Book-Balancing Magic Has Nowhere to Hide: Source-Native Accounting and Actual Continuation

Source-Native Whole-Ledger Accounting

Jian Gao

Independent researcher · innersummer@hotmail.com · ORCID 0009-0009-5002-1655

Every difference has an entry. Every settlement has a receipt.
The ledger does more than hold a grudge. It generates the next entry, pays, answers, and turns the page.

Abstract

We construct a whole-ledger continuation chain fulfilled by one actual source: an original residual in the complete future model generates the syntax and responsibility row of the next request; the source program executes and strictly pays the same debt; and its completion event delivers an answer, a receipt, a complete ledger write, and the actual successor. With the source and lawful compilation contract fixed, the named candidates based on event trimming, receipts from another source, hidden responsibilities, equal-budget migration, and zero-budget pseudo-settlement cannot construct the required lawful objects. The construction holds for the original source factory, initial material, configuration, and every natural-number-indexed admission stop in its cofinal runtime sequence, retaining future readings at every allowed action word and site. Original syntax, paid state, the full execution trace, and responsibility ownership enter the same next write. Zero numerical effect does not erase syntax payment, and a new responsibility receives its own source rather than refilling the old debt. Complete event inventories and complete open-responsibility fibres are the accounting objects; finite explicit rows answer the whole ledger through exact remainders, while split and merge preserve every parent, child, occurrence multiplicity, and original typed receipt. For a fixed original debt, finite non-refilling histories enclosing one strict payment make the payment macro-step relation well founded without a total closure law. A separate total law emitting exact settlement or an actual paid macro-step at each current constructs a finite settlement history. Final settlement requires both the original terminal receipt and an empty same-debt fibre in the actual successor. Target six exhibits actual repair, payment, and settlement; an original integer consumer cannot be recovered by any function from the entire family of finite complex observations; and, conditional on the specified right-half-plane zero receipt, four traces of a finite window, inverse residual, two means, and a cochain persist into a correction program, its actual new inventory, and every later reading. The ledger both governs how responsibility ends and generates how it continues to work.

Keywords: source-native whole ledger; complete event inventory; syntax payment; responsibility lineage; actual answer; complete future model; typed settlement

Contents

Abstract 1

1 The Balance Is Zero. Where Is the Settlement Receipt? 3

2 The Whole Ledger Is Bigger Than This Screen 4

3 A Page Turn Cannot Make Responsibility Disappear 7

4 The Same Zero Can Hide Two Different Ledgers 11

5 Move the Books. The Debt Does Not Get a New Life. 13

6 This Is No Migration: From $2+4$ to $3+3$ 16

7 No Discount for the Final Value: Syntax Pays Its Own Bill 18

8 The New Ledger Takes Over the Old Ledger—and the Next Request 21

9 The Original Ledger Speaks: Request, Payment, Answer, and the Next Occurrence 25

10 One Real Payment. No Amount of Moving Can Refill It. 34

11 Two Keys to the Settlement Door 35

12 Change the Language. Bring the Old Ledger. 38

13 The Whole Ledger Also Produces Mathematical Consequences 40

14 Long Computations Do Not Get to Abandon Their Ledger 45

15 The Exact Connections to Existing Methods 54

16 The Ledger Does More Than Hold a Grudge. It Pays. 56

Appendix A Shared Theorems and This Paper's Independent Proofs 57

Appendix B Contracts and Units 58

Code and data availability 58

References 59

1 The Balance Is Zero. Where Is the Settlement Receipt?

Writing zero on the last line of a ledger is easy. The hard part is explaining what has vanished: the debt, or its column.

Suppose someone reports, “Done.” Ask about an unfulfilled commitment, and they open a new interface: the item is gone. Ask about the original bearer, and they produce a transfer slip. Ask about the recipient, and they display a consolidated report: debit and credit cancel, and the balance is zero. A transfer can be lawful, and numerical cancellation can be correct. They still owe an answer to the same question: what actual disposition did the original responsibility receive?

Source-Native Whole-Ledger Accounting returns that question to the actual occurrence. The source fixes the event family, compilation law, complete responsibility ledger, and reading ports in advance; the same occurrence generates the finite processing fragment, each row's destination, and the original successor. “Unforgeable” has a precise type semantics here: with this source and contract fixed, each named unlawful candidate admits a function from its required qualification type to the empty type. Trimming, concealment, restructuring, equal-budget payment, and false settlement will each give this definition its concrete force.

Yet a ledger that only exposes lies still lacks its most useful ability: **can it use the material it has already generated to do the next thing itself?** Our answer is yes. The complete future residual enters the original root and grows into the syntax of the next request; executing that syntax pays the same mathematical responsibility; actual completion delivers the answer and original receipt; and the original whole ledger passes every old item, the new item, and the paid material to the same actual successor. Payment is not a new name for a numerical value, and the answer is not a medal pinned to the ledger.

Main Theorem 1.1 (The original ledger generates a request, payment, answer, and continuation). Fix any original source factory $\mathcal{F}$, complete initial frame I, program cfg, and language identity $\mathsf{cfg.LowVar} = X$. The runtime generates admission stops indexed by $n \in \mathbb{N}$, cofinal in its actual ticks. At every such stop, the next complete packet generated by the native birth program and the original root current satisfy the following contract.

1. The literal residual and paid relation boundary have the same semantics in every environment; their relation is also retained in the complete future model at every allowed action word and site. The entire material, execution trace, and responsibility ownership are transported along the actual current equality, and the next request is generated from this material.
2. The old responsibility row and the born row both read the complete ledger destination of this original write. The syntax of the next request produces positive remaining credit; its actual payment branch supplies the unique same-debt target, strict budget debit, no refill, and the destination of that same whole-ledger write.
3. The original suffix execution completes this request, with completion value equal to the original material's effect. The original stock target compiles the actual settlement action into responsibility admission; its dependent eliminator returns the original answer, receipt, old and new rows, and canonical successor together.
4. Section 14 gives a conditional finite-window and inverse-residual construction for every specified original analytic input. Four old traces, the next window inventory, and the correction

trace enter this actual continuation; the correction request, actual born packet, and all later observations consume the same write.

Proof roadmap. Sections 7–8 establish syntax, responsibility birth, typed inventory, and the complete cofinal interface of one runtime. Section 9 proves (1)–(3) in full; Section 14 proves the conditional long-computation contract in (4). Each "next" is generated by the original compiler. No step chooses a new syntactic representative from a future quotient class. Final settlement is defined separately by the two conditions of an original terminal receipt and no same debt in the successor; Sections 10–11 respectively prove payment well-foundedness and finite settlement using the total emission law.

The general residual identity, birth payment, and completion suffix hold for the actual material of any original factory. Section 9.2 additionally constructs a common-query factory $\widehat{\mathcal{F}}$ from any $\mathcal{F}$, installing a complete paired reader so that an actual query reads the old value and effect of the same current. This additional reader is generated within the source; the general chain does not assume its installation.

Target six and the original integer consumer provide two independent instances. Target six repairs $2 + 4$ into $3 + 3$, pays the same-debt budget $1 \to 0$, and then supplies an independent terminal ledger and a successor with no same debt. On another original arithmetic chain, the same nonzero integer class becomes zero in every finite-stage complex observation, while the original integer consumer reads a nonzero value: no function can recover that consumer from the entire observation family. The long-computation section gives a separate conditional application: for every specified input, the residual of the old write becomes the next correction program's input, with the window reading and complete computation history retained together.

The shared results on process identity, representation, and source execution are cited by name from the published second edition of the process core [6]. This paper independently proves complete event admission, rowwise whole-ledger accounting, responsibility restructuring, the distinction between payment and settlement, and the passage from the original future model to an actual request, actual payment, completed answer, and long-calculation continuation. Appendix A locates the theorems and their division of responsibility; the proof supplement gives the detailed compilation relations and individual proofs.

2 The Whole Ledger Is Bigger Than This Screen

2.1 Give Responsibility an Identity Before Discussing the Reading

Definition 2.1 (The complete ledger and source occurrence). A responsibility world supplies a type S of support positions, a responsibility type R, and a dependent type $\mathrm{Open}(s, r)$. A value $w : \mathrm{Open}(s, r)$ witnesses that this responsibility is actually open at that position. The complete live ledger at position s is

$$\mathcal{L}(s) := \sum_{r:R} \mathrm{Open}(s, r). \tag{1}$$

Both the semantic claim $c(e)$ and natural-number budget $b(e)$ of an entry $e = (r, w)$ are read from w; a position also has an anchor, occurrence association, and lineage key. Two witnesses supporting the same budget reading need not be the same ledger entry. “Complete” is not a compliment paid to a table: $\mathcal{L}(s)$ is the entire dependent fibre. There is no field that lets a consumer tick off a few selected responsibilities.

A source fixes a current type C, an initial current, a per-current event family $E(x)$, a map from events to support positions, and two compatible compilers. The structural compiler sends $\varepsilon : E(x)$ to one of five forms: native write, relation write, continued transport, redirect, or local terminal ledger. The payloads of the first four determine their successors; a terminal ledger has no local successor. From that same ε, the whole-ledger compiler generates the evolution of the complete ledger and a generated processing fragment. An emitter $g(x) : E(x)$ installed afterward must also satisfy the following condition: the target occurrence given by the compiler equals the occurrence emitted by that emitter at the target current. An arrow does not permit an event swap halfway through.

The source also fixes its reading ports in advance. For a port p and event ε, the outcome type is

$$\mathrm{Read}(p, \varepsilon) = \left(\sum_{a : \mathrm{Active}(p, \varepsilon)} \mathrm{Payload}(p, \varepsilon, a) \right) \sqcup \mathrm{Inactive}(p, \varepsilon). \tag{2}$$

Classification, activation witness, and payload are dependent source outputs. “Inactive” therefore has its own exact witness, rather than an empty value inserted at whim. A port installation consists of an injective embedding and a commuting equality for the entire outcome. It preserves the branch and the witness carried by that branch, as well as its numerical value.

This immediately constrains subsequent consumers: for the same complete source, event, and installed port, two installation paths cannot read different dependent outcomes. The proof first compares the source classification. In the active branch, injectivity of the sum constructor gives the same activation witness, and the payload is transported along that equality; the inactive branch is similar. The installation equations reduce both paths to this original outcome. Conversely, a Boolean construction does give two different complete sources the same lower root reading but different ports: after forgetting the port law, “they look the same” is precisely what fails to suffice.[1]

2.2 Do Not Declare the Ledger Terminal by Deleting the Next Page

Definition 2.2 (Complete event-inventory admission). Let $\mathcal{S}_*$ be the concrete source to be presented, and let $\mathcal{S}$ be its authoritative presentation. Admission includes a constructive two-way presentation of the currents and of the complete event family at every current: the forward and reverse composites are both identities. The presentation must also commute with the initial current, structural compilation, structural branch, optional successor, support position, complete whole-ledger writeback, and actual generated fragment. If the source has cofinal boundary events, the boundary event family, whether emission occurs, every predecessor, and the landing are likewise preserved in both directions. This two-way condition concerns the complete family, rather than the subset selected at this tick.

[1] The proof uses classifier equality and installation commuting identities. The Boolean counterexample changes the complete source, rather than selecting a different result under a fixed source.

Finally, admission carries an independent **terminal-exhaustion condition**: if a presented event compiles to a local terminal ledger, the complete ordinary outgoing-edge family at the corresponding concrete current is empty. An ordinary outgoing edge consists of an actual source event together with the continuing successor obtained by compiling that event; it is not an arrow a port reports as present. Including this condition in the contract licenses the passage from a local terminal ledger to terminal authority over the complete inventory. A cross-root continuation after the terminal ledger is generated by another continuation law fixed by the source in advance; it has a different type from a local ordinary outgoing edge.

Theorem 2.3 (A terminal ledger without trimming). Under the admission of Definition 2.2, every concrete source event can be recovered by a round trip. A faithful terminal ledger in the presentation corresponds to an actual terminal ledger of the concrete source compiler, retains the original whole ledger and generated fragment, and excludes every ordinary outgoing edge at the corresponding concrete current.

Proof. Every event may be present while a door remains open beside the terminal ledger; the decisive part of the proof comes last. The two inverse equalities give round-trip recovery of events directly. Suppose the presented event is terminal. The successor equation makes the successor of the corresponding concrete event absent as well. Expand each of the concrete compiler's five constructors: the four continuing constructors all have a present successor, contradicting absence; only the original terminal constructor remains. The whole-ledger and fragment equations give equality of the complete dependent objects. Finally, terminal exhaustion makes every ordinary outgoing edge absent. The event bijection alone guarantees that no event was deleted; excluding outgoing edges beside the terminal ledger does use the final condition. ■

Counterexample 2.4 (What was cut away was an exit). Take a concrete event family at a single current with two distinct constructors: a terminal event t and an actual write event w. The compilation of w carries an actual successor. Compare it with a singleton presentation retaining only the terminal event. If this were a complete presentation, its forward map would send both t, w to the unique element; the identity reverse composite would then give $t = w$, contradicting the distinct constructors. Even if only the selected t is required to compile to a terminal ledger, w directly refutes terminal exhaustion. The objects of this counterexample are the mixed event source and its trimmed presentation.

Deleting a responsibility and deleting an opportunity for action use the same manoeuvre here: first shrink the universe, then declare everything settled in what remains. Theorem 2.3 prevents that manoeuvre from passing type checking.

2.3 Occurrence Identity Is Not a State Label

Proposition 2.5 (Registered occurrences and locating differences). An actual visit consists of a current together with the source-generated history of arrival. The history begins at the source initial current and proceeds step by step along the original compiler's successor equalities. A history with a cofinal landing also retains the original boundary event and its predecessor flow. A registered occurrence is

$$\mathsf{Occ} := \sum_{v:\mathsf{Visit}} E(\mathrm{current}(v)). \tag{3}$$

Two occurrences at different visits cannot be identified merely because their current labels agree. For a fixed source-generated process, locating a registered occurrence at a visit of that process together with its original event supplies a positive internal structural identity. Thus, in this registered-occurrence semantics, every actual real difference that can be stably referred to can be located; it cannot simultaneously claim to be nonredundant and to have no internal structural identity.

Proof. The same current can be reached twice; the arrival history is what separates those visits. Taking the first projection of an equality of dependent pairs gives equality of visits, so different causal histories cannot be erased. The state embedding of the source-generated process is injective on the complete root and visit; the inverse registration of the process recovers a state agreeing with the original visit. This state, visit, and original emitted event construct the structural identity. More abstractly, if an actual difference is classified as either redundant or structurally identified, an alleged external difference that is nonredundant and internally undetermined contradicts nonredundancy in the first branch and the lack of internal determination in the second. The positive locating construction comes first; exclusion of an unaccounted difference is its consequence.

This is the first meaning of "every difference has an entry": an unintelligible difference cannot be deleted. Within the declared actual source it must have a recoverable position. The second meaning follows immediately: at that position, whose responsibility remains open?

3 A Page Turn Cannot Make Responsibility Disappear

3.1 Original Admission Leaves a Name; Disposition Leaves a Receipt

Definition 3.1 (Admission, live responsibility, and terminal records). An admitted responsibility retains its original admission receipt, as well as current content, remainder, bearer, beneficiary, and scope. The original receipt records the source event, source anchor, responsibility occurrence association, and lineage, together with proof that this source has authority to generate this responsibility. A need, promise, commission, task, dependency, or risk can provide the starting point for admission; that starting point must still connect to source authority, acceptance or authorized assumption, and jurisdiction over discharge. A need alone does not generate someone else's obligation.

A live responsibility can receive progress, maintenance, deferral, scope adjustment, and accepted transfer; each disposition must carry the witnesses required by the original contract. Narrowing the scope requires the corresponding jurisdiction, and transfer requires evidence of reception. Supersession retains an actual successor responsibility covering the original obligation; bearer release retains a lawful target responsibility or a higher-level disposition path. Neither silently deletes the old responsibility.

A terminal receipt has five distinct lawful forms: fulfilled completion, authorized waiver, invalidation of the original responsibility, abandonment with consequences borne, and accepted unrealizable residue. Their grounds for ending responsibility differ, and they cannot all be renamed a numerical payment. The terminal record retains the complete responsibility being ended, the source event, and its terminal receipt. Reopening starts from this record, takes a new actual

admission as input, and preserves the original lineage; it does not issue a new identity saying "never owed."

Theorem 3.2 (No birth from nowhere and no silent disappearance). On every actual edge of any finite lifecycle history generated by these native edges: a new slot that was absent and then becomes live has an actual admission receipt; an existing position can become live from a nonlive state only through admission or reopening; and when a live position becomes nonlive, the actual terminal receipt for the original responsibility at that position must be retained.

Proof. Each edge changes only one position, so it suffices to inspect, edge by edge, what can make a position light up or go dark. First exhaust the single-edge cases. Admission appends a live responsibility carrying its original receipt to the slot sequence; reopening acts only on an existing terminal record. Every other edge carries the live position it acts on: if the position under consideration differs, its value is unchanged; if it agrees, progress, maintenance, deferral, narrowing, transfer, supersession, and bearer release still produce a live responsibility. Only the termination constructor replaces it with a terminal record carrying the original responsibility and terminal receipt. The three conclusions can therefore come only from admission, admission or reopening, and termination, respectively. Induction by concatenation on a finite history retains the actual edge that first changes the position's status, together with its receipt. ■[2]

A concrete promise admission turns the empty state into a live responsibility with remainder 3; its accepted transfer changes the bearer to a successor while the slot remains live and the lineage unchanged. Another world rejects admission: needs can be proposed, but both the acceptance type and the authorized-assumption type are empty, so there is no admission value. This is not a failure to find consent; its specified admission type is proved empty.

3.2 A Few Rows Changed This Tick Does Not Mean the World Has Only a Few Rows

Definition 3.3 (Whole-ledger evolution and generated fragments). For a continuing event $\varepsilon : s \to t$, the evolution of the complete ledger has two total outputs:

$$d_\varepsilon : \prod_{e:\mathcal{L}(s)} \sum_{f:\mathcal{L}(t)} \mathrm{Evolve}_\varepsilon(e, f), \qquad o_\varepsilon : \prod_{f:\mathcal{L}(t)} \sum_{e:\mathcal{L}(s)} \mathrm{Evolve}_\varepsilon(e, f). \tag{4}$$

The first answers where every old responsibility goes; the second answers where every target responsibility comes from. They are not stipulated inverse bijections: splitting and merging require many-to-one and one-to-many associations.

Equation (4) is the ordinary continuation contract within a fixed responsibility world. Responsibility birth and expression revision first generate a new responsibility world and its initial ledger from the source, then apply this contract to the complete fibres of that world. A new responsibility therefore begins its first write in an initial ledger that already contains its born row. Native admission still retains the source occurrence of Definition 3.1. A local terminal ledger instead supplies $\prod_{e:\mathcal{L}(s)} \mathrm{Discharge}_\varepsilon(e)$: a nonempty ledger requires a receipt indexed by every original entry, while an empty ledger permits vacuous satisfaction. The complete lifecycle interface in Section 5.2

[2]The shared foundation is Proposition 6.1 of [6].

includes two sum types: every old entry has an actual descendant family or terminal receipt, and every new entry has an actual ancestor or admission from this occurrence.

The compiler does not receive a retrospective, manually completed instance of (4). It receives the source occurrence, generates finite explicit rows and their coverage scheme, and folds them into (4). There are exactly three coverage schemes for a continuing fragment. An **in-place remainder** applies at the same support position: a successful query returns a row index with an equality saying "this is the very entry queried"; a miss carries the original entry. **Complete finite coverage** gives total maps from source and target ledgers to finite row indices and proves that every index really returns the queried entry. A **transport remainder** permits finite exceptions across positions, but every missed item must be given to the same source-generated remainder event, whose compilation already determines all destinations and origins.

Terminal ledgers have two separate generation schemes: an exact index over the complete responsibility fibre into finite terminal rows, or a support-discharge occurrence actually selected by the original terminal source. The latter applies the support receipt compiled from that same source to the entire fibre, indexed by original entries, without assuming finitely many entries. It does not accept an arbitrary support receipt supplied after the fact.

Theorem 3.4 (Finite presentation, whole-ledger answers). Every continuing fragment above folds into the complete evolution of (4); both terminal-fragment forms fold into complete itemwise termination. The result equals the whole-ledger compilation of the same source occurrence. An entry missed by an in-place remainder is carried as that exact original entry, with no concealed transfer. Complete finite coverage cannot disguise an actually infinite responsibility fibre as a finite list. A transport remainder allows disposition of the complete fibre beyond its finite exceptions.

Proof. The hard cases are the items omitted from the list: they have no explicit row index, yet must have definite destinations. In the in-place case, inspect the optional index for each item: on a hit, transport the generated row along the attached entry equality; on a miss, return this entry and its identity-carry witness. The disposition constructor in the miss branch is not transfer. In complete coverage, take the specified destination index for any old item and the specified origin index for any new item, then obtain the generated rows along the coverage equalities. These equalities make the maps from finite rows onto each complete fibre surjective, so this coverage form requires the fibre itself to be finite. In a transport remainder, the hit branch is as before, and the miss branch directly calls the source-generated remainder destination or origin. Finite terminal rows yield each terminal receipt along the equalities of the complete index; support discharge applies the exactly selected support receipt to any original entry. Finally, the fragment is type-indexed by this whole-ledger compilation result. Fold equality is part of the compilation contract: a different aggregate table cannot be substituted after folding. ■

A concrete payment root uses precisely such a coverage scheme. The source-generated gated-claim payment root takes an actual strict advance and the exact whole-ledger terminal receipt at its target as input. Its payment fragment has **zero explicit rows**, and every query selects the remainder; yet the remainder event compiles to the complete ledger evolution of that original advance, and its fold equals the original payment evolution entry by entry. The gated payment for target six uses this construction. Zero here counts explicit exceptions, not responsibilities, and

certainly does not mean "no payment required." This instance and the target-six action chain in Section 6 retain their respective payment objects.

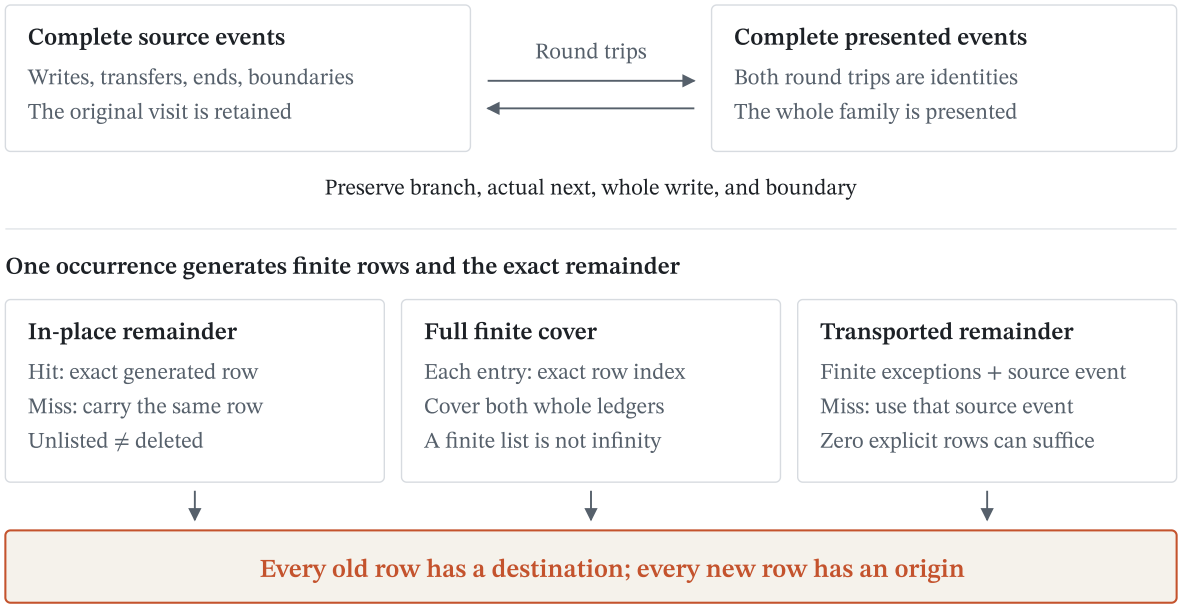


Figure 1 Complete event inventory and the fragment processed this tick are two different questions of completeness. The two-way presentation above removes no event; finite rows with exact coverage below answer the entire responsibility fibre.

3.3 Ask About an Item; Get an Answer About That Item

Theorem 3.5 (Itemwise disposition and the same successor). Fix an admitted source-generated root, a registered visit, and its whole-ledger occurrence. For every entry of that complete live ledger, the original compiler supplies either exact evolution to a target entry or that entry's terminal receipt. In a continuation, the target entry lies in the complete ledger of the successor event actually emitted by the same root; if generated causal authority for the entry is already held, successor authority is derived from the same actual evolution. The "answer-and-next" of any two entries reads the same actual successor current.

Proof. The danger is that the target entry lands in a similar-looking ledger that is not the actual successor's ledger. Expand the original whole-ledger compilation result. Each of the four continuing constructors supplies a target occurrence and a destination from (4). Injectivity of the local-successor equality identifies the target current with the actual successor being queried; the emitter equation identifies the target occurrence with the event actually emitted by the root at that current. The target entry transported along both equalities is therefore not a candidate from another similar ledger. The terminal constructor directly calls the itemwise receipt of complete terminal evolution. Initial causal authority comes from the original generated row selected by the same coverage selector: merely storing a row in inventory is insufficient. Successor authority is derived from the original authority and the exact successor equality just proved, without fresh admission from the target side. The successor of "answer-and-next" is the compiled reading of the original occurrence and does not depend on which entry is queried. ■

Proposition 3.6 (Actual disposition forces the exact generated row to appear). If this disposition of the original complete ledger is termination, or an entry is actually transferred in the original folded evolution, that disposition fact generates canonical row authority for the original entry. The target and dependent evolution in a continuation, or the receipt in termination, are exactly those used by the whole-ledger fold. No downstream row index or inventory-membership proof is required.

Proof. The key case is the in-place remainder: a miss can only carry, so an actual transfer must be a hit. Termination uses the full-coverage selector of finite terminal rows or the source-generated selection of support discharge. For continuation, inspect the fragment type: complete finite coverage has already fixed the row index; a transport remainder generates exception authority on a hit and remainder authority on a miss, each from that same selector. If an in-place remainder missed, its folded result would be a carry constructor, distinct from the assumed transfer constructor, so it must hit. Finally, transport along the equality between the fragment fold and the original whole-ledger compilation to obtain target equality and heterogeneous equality of dependent evolution. ■

Every registered responsibility receives an exact answer, including entries absent from this tick's list. We can now ask what that screen showing only totals has hidden.

4 The Same Zero Can Hide Two Different Ledgers

Definition 4.1 (Observation safety relative to a consumer). Let A be the complete state type to be observed, $q : A \to O$ the observer, and $c : A \to D$ the consumer of interest; a dependent disposition can be packed into D together with its index. We call q safe for c if

$$\mathrm{Safe}(q, c) :\Longleftrightarrow\ \forall x, y : A,\ q(x) = q(y) \Longrightarrow c(x) = c(y). \tag{5}$$

A hidden-difference witness is a pair of states carrying $q(x) = q(y)$ and $c(x) \neq c(y)$. Safety is never an absolute virtue of an observer: the same port may suffice to compare balances while failing to decide whether an original responsibility has been released.

Theorem 4.2 (A hidden difference cannot be certified safe). Given a hidden-difference witness, $\mathrm{Safe}(q, c)$ is empty. Under any further compression $h : O \to O'$, the same states remain a hidden difference for $h \circ q$ relative to c. If an actual factorization $c = \bar{c} \circ q$ has separately been supplied, then q is safe for c.

Proof. Substitute the witness's $q(x) = q(y)$ into (5), contradicting $c(x) \neq c(y)$. Further compression only applies h to this observation equality; the consumer inequality remains. With a factorization, apply $\bar{c}$ to $q(x) = q(y)$ and use the factorization equations at both endpoints. Safety here means constancy within fibres. It does not silently add the choice needed to construct a decoder: factorization is sufficient, while its general converse construction belongs to a separately declared classical-choice adapter. ■

Instance 4.3 (A hidden admission that actually occurred). Return to the actual promise-admission edge of Section 3. A thin observer filters slots by a scope that never displays this responsibility; it returns an empty list both in the empty state and after admission. The

complete current-responsibility observer reads the empty slot sequence before admission and the admitted live responsibility afterward; length alone distinguishes them. The original admission event, invisibility equality, and this difference in complete responsibility state jointly generate a redirect witness retaining the actual change hidden by the thin observation.

When the interface says “no change,” there are thus three possible conclusions: it does suffice for the current consumer; a hidden difference has refuted it; or classification has not been obtained. The third is not a softer version of the first. Whole-ledger accounting allows judgment to remain open, but it does not package missing evidence as safety.

As soon as this actual hidden-admission edge is present, the consumer-safety type is empty. The same criterion will return in the algebra of Section 13: every finite-stage complexified observation becomes zero, while the original integer consumer explicitly reads a nonzero value.

4.4 The Original Receipt Also Delivers Complete Recovery or an Exact Residual

Proposition 4.4 (Four mathematical dispositions of the same original transfer). The receipt of the original actual native write generates its own complete history, relations, and evaluator. Its source mathematical disposition supplies one of four branches: complete linear recovery; a nonzero evaluation of an actual relation; a nonzero kernel element in the original completion carrier; or an exact representative outside the image of the original evaluation map. The last branch proves, for every recovery function, that this representative cannot be recovered.

Proof. First eliminate the receipt from the original compiler's actual write branch, obtaining the generators, seed, continuation structure, and evaluator of the same occurrence. Let E be its linear extension on the complete free word module. If E reads an actual relation as nonzero, return that relation and its nonzero image. Otherwise the relations are sound, and E descends to $\bar{E}$ on the completion carrier. If its kernel is nonzero, return a nonzero kernel element with zero evaluation; if the kernel is zero and the image complete, the original construction generates a complete linear equivalence and its inverse. In the remaining coverage-failure branch, take a representative a of a nonzero cokernel class, obtaining $a \notin \operatorname{im} \bar{E}$. For any function f, if $\bar{E}(f(a)) = a$, then $f(a)$ would give a preimage of a, a contradiction. Proof Supplement A gives the concrete relations, kernel, and cokernel in each branch. This original classification is a noncomputable mathematical disposition; syntax execution fees of the original source program still use its own receipts. The original transfer row, whole ledger, and literal successor are retained together; a new residual port does not impersonate strict payment. ■

5 Move the Books. The Debt Does Not Get a New Life.

5.1 The Bearer Relation Moves; the Debt's Past Does Not

Definition 5.1 (The same site and the same debt). For a source entry e and target entry f, being at the same site requires equality of anchor, occurrence association, lineage, responsibility label, and semantic claim. Being the same debt fixes only the lineage and semantic claim determining the continuing obligation:

$$e \sim_{\text{debt}} f :\Longleftrightarrow \lambda(e) = \lambda(f) \text{ and } c(e) = c(f). \tag{6}$$

A lawful change of bearer, anchor, or local clock can therefore preserve the same debt. A target entry is genuinely new relative to the complete old ledger only with a refutation $e \sim_{\text{debt}} f \to \bot$ for every old entry e; "it differs from one old entry" is insufficient.

Proposition 5.2 (Continuation does not reissue budget). Original whole-ledger row evolution has three forms: carry preserves the same support position and exact entry; maintenance preserves the same site and strictly decreases budget; transfer carries an actual transfer receipt, preserves same-debt identity, and bounds the target budget by the original budget. Thus every lawful row evolution satisfies

$$e \sim_{\text{debt}} f, \qquad b(f) \leq b(e). \tag{7}$$

Carry has equal budgets; maintenance has strictly smaller budget; transfer may have smaller or equal budget.

Proof. Eliminate the three row-evolution constructors separately. In carry, support-position equality and heterogeneous equality of entries rewrite the target to the original entry, making both conclusions reflexive. Maintenance directly supplies the lineage and claim equalities and strict inequality, which implies nonincrease. Transfer supplies its lineage and claim equalities and no-refill field. ■

Equality in transfer deserves to stand on its own. It is not a failed proof of lawfulness: it is a lawful case in which payment has yet to occur. A change of bearer, directory, coordinates, or clock can explain why the responsibility now appears elsewhere; it cannot explain why that responsibility has been discharged. A renaming receipt likewise preserves the original occurrence association and lineage; an unaccepted transfer proposal does not even change the bearer.

5.2 Every Piece Still Needs a History After the Split

Definition 5.3 (Split and merge with itemwise coverage). A split receipt fixes a source event owning the original responsibility occurrence, a nonempty child list, and a whole-coverage witness. Every child has a descendant relation from the original parent, anchor transport, same-lineage proof, and preservation of local discharge. A merge receipt fixes a nonempty parent list and a target entry, retaining for every parent its ancestor association, anchor transport, same-lineage proof, and preservation of local discharge. Whole coverage cannot replace any of these itemwise witnesses.

On a complete root ledger, these receipts are further indexed by the original event and whole-ledger evolution. If two distinct target entries have the same original origin, the original restructuring compiler must generate a split receipt covering that origin fibre: every listed child is in the target ledger, and every target-ledger entry in that origin fibre is covered. Each child's budget is strictly below its parent's. If two distinct old entries have the same target, an equally exact merge receipt must cover every original parent in the target fibre, omitting none and adding no phantom parent. If the two entries were already equal, the compiler may return the identity branch rather than counterfeit a split or merge event.

Theorem 5.4 (Totals cannot replace responsibility coverage in split and merge). Under the source-fixed restructuring contract of Definition 5.3, a nontrivial split retains every actual child's origin and local discharge conditions, and a nontrivial merge retains every actual parent's local discharge conditions. Equality of child counts or totals cannot replace these dependent receipts. In particular, a zero-budget parent admits no lawful nontrivial split.

Proof. Equal totals say nothing about any item's origin; coverage and membership conditions do the work. Analyze the classification of the source-fixed compiler in its two branches. The identity branch gives equality of the dependent entries compared, so it cannot apply to distinct entries. The split branch uses complete coverage of the actual origin fibre to send any actual child into the receipt's child list, then reads off its descendant, lineage, and local-discharge witnesses. The requirement that every listed item belongs to the actual target ledger excludes fictional items used to fill coverage. Merge uses the symmetric coverage condition for actual parents, retaining each discharge witness. At zero parent budget, any actual child would require $0 \leq b(\text{child}) < b(\text{parent}) = 0$, a contradiction. ■

Strict decrease is asserted for each child individually; it cannot be replaced by a decrease in the sum of all child budgets. The conserved object in restructuring is first the responsibility association and discharge coverage, not the number of slips or an arbitrarily summed quantity.

Counterexample 5.5 (Two rows can still launder away a row). The old and new entries of the original weak restructuring construction are Boolean-indexed, so each side has two entries; yet both destination and origin maps are constantly `false`. Every selected pairwise transfer can supply $0 \leq 0$, and the row counts agree exactly. Nevertheless, targets `false` and `true` have the same original origin, while old entries `false` and `true` have the same target. Returning identity from complete classification would equate the Boolean entries; returning split or merge would require the coverage type deliberately left empty in this construction. Both routes contradict the data. This is a counterexample to upgrading weak data into complete restructuring certification.

The two-sided ledger in the complete lifecycle checks both ends accordingly: every old entry has either a nonempty covered descendant family or a source-owned terminal receipt; every new entry has either an actual ancestor or a genuine admission from this source event. The associations at both ends must also agree. Eliminating these two sum types gives, respectively, no silent sink for an old entry and no free source for a new one. This is stronger than counting how many slips remain before and after: slips can split; responsibility cannot evaporate in the process.

5.3 Original Restructuring and the Complete Remainder Enter Actual Execution

Theorem 5.6 (Original restructuring certificates lift with calculation). Fix the original authoritative root, the complete native program read from its actual compiler, and the original registered input. Appending mathematical execution preserves the complete typed fields and original split/merge certificates of every old responsibility: children, parents, source ownership, anchor transport, lineage, local discharge, complete coverage, exclusion of phantom members, and strict debit for each child all enter the same actual appended write. If a per-current complete continuing-payload program generated by the original source law is also given, the finite mathematical rows and same-source complete transport remainder of a general continuing branch fold into the whole join of the original whole ledger and mathematical row; the actual tick directly consumes this certificate, all old destinations, and the original successor.

Proof. Construct the responsibility type with two disjoint tags, old and mathematical. The old-responsibility embedding j has a reading left inverse q, satisfying $qj = \mathrm{id}$ field by field: original admission, authority, assumptions, and discharge objects are read back, rather than merely a responsibility identifier. Complete original destinations and origins are transported under j; the mathematical row associates only mathematical tags. Repeated origins or destinations on the old side therefore reflect exactly the original compiler's restructuring problem, and the mathematical side creates no cross-fibre with it.

When the original classification is identity, lift the dependent-entry equality along the left inverse. For split, map the original children list through j item by item. Membership, complete origin-fibre coverage, and exclusion of phantom members are preserved in both directions through $qj = \mathrm{id}$; transport descendants, anchors, lineage, local discharge, and strict budget inequalities individually. Merge transports all original parents in the same way. The target thus receives the image of the original mother certificate, rather than a separately submitted classification certificate.

Given the original source law's per-current complete continuing-payload program, eliminate the original compilation result for any actual native, relation, continued-transport, or redirect payload, obtaining that branch's own whole-ledger evolution, generated fragment, and successor. In the complete join, old destinations are the original destinations, and mathematical destinations are those of this actual execution step; origins are analogous. The finite fragment explicitly retains only the mathematical row, while every missed old entry is handed to the original remainder event just eliminated. Expanding the fold entry by entry in the complete old/mathematical Sigma branches gives the join's complete destinations and origins. The original restructuring proof lifts to this fold by the preceding argument; the compilation equality of the same tick then identifies its certificate, destinations, and successor with the actual compilation result. ■

Complete native finite installation also has a direct positional reading: if there are m old rows, the new inventory has $m + 1$ rows, each old position enters through the original final-position extension, and the mathematical row occupies the new last position. Repeated claims and identical payloads are not deduplicated. The original identity certificate makes both old endpoints injective, while disjoint old/mathematical tags prevent the new row from colliding with them; general split/merge uses the original restructuring certificate above. The complete-remainder form does not require a finite old responsibility fibre, and finite row counts do not pay the coverage

obligation.

Restructuring is now more than a static permit: all the Type receipts already paid by the original mother source follow the original program into the actual next tick. Section 7 uses this whole-ledger square to let a syntax responsibility without an old bearer pay for itself while preserving the old ledger.

6 This Is No Migration: From $2+4$ to $3+3$

6.1 Let the Original Action Occur, Then Read Its Answer

The original target-six process starts from $2+4=6$. The right-hand 4 is not prime, and its actual least-factor reading is 2. The original production compiler's finite factor-repair inventory contains the structural key of this repair. Its "complete" branch is therefore excluded by the nonprimality of 4, and its "no repair available" branch by the inventory-membership witness. What remains is the action branch already generated by the original compiler.

The action changes the left side to $2+(2-1)=3$ through the original repair channel, and the total target 6 fixes the right side at 3. Thus we obtain the actual target of compilation, rather than a desired answer submitted to the compiler:

$$(2,4) \xrightarrow{\text{original factor-two repair}} (3,3), \qquad 3+3=6. \tag{8}$$

The action receipt carries only the original step, the target generated by that step, and the equality identifying it as the original compiler's output. It has no free field stating "a prime pair exists," "occupancy is nonzero," or "settled."

Only then is the target read. The original target classifier checks primality on both sides: if it remained in the action branch, this would contradict its classification equality when 3 and 3 are both prime. The classifier therefore emits the actual terminal-class reading. That reading extracts the valid-prime-pair fibre, which constructs the original actuality settlement and gives a nonzero original occupancy reading. The original action and terminal-class reading are two dependent steps: the latter must read the former's exact target, and a prepared conclusion cannot be inserted into the former.

6.2 The Same Action Enters the Responsibility Ledger

Definition 6.1 (The target-six action debt). Take an open state u and a paid state v with budgets $b(u)=1, b(v)=0$. The payment step of this debt law is generated by the original action above, and its termination witness by the target terminal-class reading above. Attach this debt law to the emitted original arithmetic occurrence: the extended live ledger retains every old entry and adds this action debt. The occurrence-indexed activation source emits the original payment branch, and the direct compiler compiles that branch into a continuing evolution of the entire extended ledger.

Theorem 6.2 (The actual payment chain for original target six). The original target-six action supplies an actual strict payment of the same-debt budget $1 \to 0$, an independent whole-

ledger terminal receipt at the exact paid target, and an original arithmetic successor driven by that receipt in which the debt is no longer activated. Payment, terminal ledger, and continuation come from their respective original compilers; none is manufactured from another by saying "the budget is already zero."

Proof. Payment, terminal ledger, and continuation each have their own compiler. The easiest step to omit is the last: the world keeps moving after the terminal ledger, so the successor must be checked to see that this debt is no longer activated. First obtain the generated action-debt step from the actual action of (8). Its source, target, and budget readings reduce to $u, v, 1, 0$, giving the strict inequality $0 < 1$. The extended ledger evolution carries each old responsibility and compiles the actual step of this debt to the same debt's target entry. The emitted source-branch equality fixes it as this payment rather than an isomorphic candidate occurrence.

Next, the target terminal-class reading produces a support-termination witness at v. Taking it as input, the whole-ledger termination compiler supplies terminal receipts for every entry of the complete live ledger at the paid support. The original activation source emits the payment branch. An independent two-stage positive root installs the original payment at the open stage and the exact terminal receipt at the paid stage as a write occurrence and terminal occurrence, respectively. The event family at the paid stage has only the terminal constructor, and its ordinary outgoing-edge family is empty, so it passes terminal admission for the complete event inventory.

Finally, the terminal-continuation law fixed by the source in advance acts on this registered terminal occurrence and generates the original next arithmetic current. The debt state of its extended support is inactive. The complete target ledger has only the inheritance branch for original entries; the added-debt branch has no open witness. Any target entry belongs to the old-claim branch and cannot equal the settled debt's claim branch, so the same-debt fibre in the actual successor is empty. The old ledger remains, and the designated action debt is indeed no longer open. ■

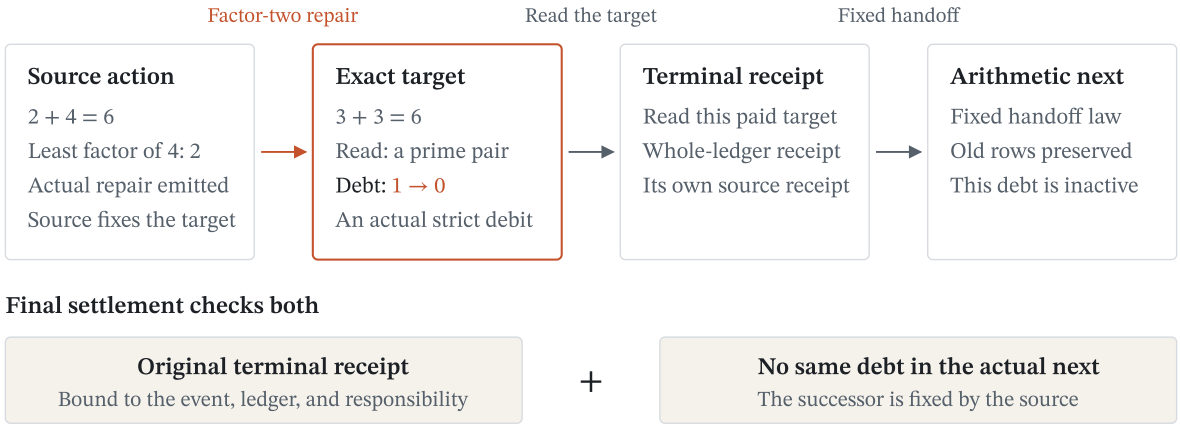


Figure 2 The original target-six chain: the original action produces the exact target, and reading that target supports an independent terminal ledger. Zero budget is no shortcut between them. The original arithmetic successor after settlement retains the old ledger but no longer activates this action debt.

Each of these four steps does its own job. $2 + 4 \to 3 + 3$ pays for a concrete action; $1 \to 0$ records its strict advance on the designated responsibility scale; the independent terminal ledger

gives itemwise terminal receipts at that support; and the successor check confirms that the action debt has not moved elsewhere and remained unpaid. Delete any step, and the remaining numbers may still look good, but the settlement contract is incomplete.

7 No Discount for the Final Value: Syntax Pays Its Own Bill

7.1 Without an Old Bearer, the Program Still Has Its Own Responsibility

Fix an admitted complete root, its original occurrence o, and the environment–expression pair (ρ, e) read from o. Write $r(e)$ for the expression's original remaining count; every original execution step removes one syntax node and decreases that count by exactly one. This unit applies throughout the section: the nodes occupied by linear, bilinear, and other primitives are counted in the original syntax; their internal arithmetic or bit operations remain measured by the respective primitive contracts.

Definition 7.1 (Paid states and relation receipts). A paid state is $x = (p, t)$, where $t : e \longrightarrow p$ is the complete execution trace already occurring in the original environment ρ. It retains the original expression, current expression awaiting execution, environment, every intermediate step, and source; its budget is $r(p)$. The original executor generates an actual step at a nonconstant expression and a mathematical settlement at a constant. Each step's relation word is the difference of its endpoints, and the relation boundary of the entire trace is

$$\partial t = [e] - [p]. \tag{9}$$

If $p = \mathrm{const}(v)$, soundness gives $v = \mathrm{ev}_{\rho}(e)$. At this completed state, for any environment increment δ, the update law reads

$$\mathrm{ev}_{\rho+\delta}(\partial t) = \mathrm{eff}_{\rho,\delta}(e). \tag{10}$$

The constant has zero effect in (10); the boundary is zero in the original environment, while the original effect after the update can still be nonzero.

Named shared result 7.2 (Responsibility birth and paid execution). Theorem 6.10 and Supplement S1 of the process core's second edition construct a mathematical responsibility row for any original input above, without requiring an old bearer. The state $x_k = (p_k, t_k)$ at tick k satisfies

$$r(p_k) = r(e) - k, \qquad |t_k| + r(p_k) = r(e). \tag{11}$$

For each $k < r(e)$, the actual execution step gives a strict payment and unique same-debt target on the same mathematical row. The terminal trace has length exactly $r(e)$, its completion value equals evaluation of the original expression, and the original old ledger is always retained as a complete remainder. At zero remaining count, the mathematical state does not reissue budget.

The execution fact must reach the whole ledger rather than leave paid history hanging on a separate face. The extended support is the original support times an optional mathematical-debt state; the complete live ledger separates all old entries from the new mathematical row. The fixed-support construction without an old bearer preserves every old entry's complete typed receipt unchanged; the same actual execution step sends the new row to the mathematical row at

x_{k+1}. The finite fragment lists only the new row and gives the remainder to the source-generated complete transport event. Expanding the fold branch by branch gives "original old destinations plus mathematical destination" at both ends, so the fold is exactly this complete write. The join of a general original event with calculation separately uses Theorem 5.6 to consume the original continuing evolution, retaining the carried/transferred distinction and activation coordinates entry by entry. Passing $r(p_{k+1}) < r(p_k)$ to the same-debt payment interface then obtains this payment, instead of inferring payment from the final value.

The proof of (9) likewise happens step by step: the identity trace has zero boundary; appending $p \to p'$ gives $([e] - [p]) + ([p] - [p']) = [e] - [p']$. Soundness and the environment-update law then give (10). This is where the original event, paid state, relation word, and whole-ledger target first meet. Supplement C expands the complete compilation and single-step consumer.

An original ten-node program gives a more concrete reading: inquiry node d after birth already contains $d+1$ original advances, its budget is $10-(d+1)$, and paid-trace length plus budget equals 10. The complete physical pair is read from the current original occurrence, while the pinned trace retains its original environment. At budget exhaustion, the complete residual is the current physical pair minus the original initial physical pair. It is zero exactly when those complete pairs agree. All mathematical nodes have executed; whether the physical pair has returned to its original value is a separately readable fact.

7.2 Integer Coefficients, Substitution Bodies, and Cancellation Are Charged as They Stand

Proposition 7.3 (Coefficients become syntax before they pay). For an original integer z and expression e, the source generates coefficient syntax $\mathrm{expand}(z, e)$ by repeated addition and original negation. Its cost is

$$\begin{aligned} r(\mathrm{expand}(z, e)) &= z(r(e) + 1) && (z \geq 0), \\ r(\mathrm{expand}(-(n+1), e)) &= (n+1)(r(e) + 1) + 1. \end{aligned} \tag{12}$$

Every coefficient of the complete word, every actual binding body, and every intermediate execution trace enters the original inventory; evaluation cancellation does not delete occurrence positions.

Proof. Nonnegative expansion begins at the constant zero. Adding one copy of e adds all its $r(e)$ nodes and one addition node; induction on the number of copies gives the first equality. Negative expansion executes one original negation around $n + 1$ positive copies, giving the second. The source executor generates a complete trace for this actual expanded expression, with length equal to its remaining count, while the paired language retains both old value and effect. Evaluating the binding in the target environment induces the original execution environment; in that compatible environment, substitution recurses on syntax structure: each encounter with an original variable binding reads and executes that complete binding body; at addition, linear, and bilinear nodes, the recursive subexpression traces and final step are concatenated in the original order. Telescoping the relation boundary of the same paid state gives the exact substituted relation receipt and independent completion value. Inventory indices are actual occurrence positions, with no nonzero-evaluation membership condition, so two cancelling actions remain two entries. ■

The final values $-1, +1$ of original ordered actions can cancel; their two execution points, residuals, and relation receipts do not. For a complete actor word w, let A_w act on source states, let U_w act linearly on reading values, and let M be the original source reader. Compose in execution order: $A_{\ell u} = A_u A_\ell$ and $U_{\ell u} = U_u U_\ell$. Write $R(w, x) = U_w M x - M A_w x$ for the total residual. The history retains each pre-action point x_i and local residual r_i. If the first letter is ℓ and the tail is u, transporting this local residual along the tail gives

$$R(\ell u, x) = U_u(U_\ell M x - M A_\ell x) + R(u, A_\ell x). \tag{13}$$

Induction on word length gives the total residual above, while history length remains $|w|$. Put $n = |w|$. The complete common-tree program executes one original paired program, with trace length equal to the original tree budget. The Model on original words, the complete prefix, and their joint program have respective node fees $n + 2, n + 3, 2n + 5$; the actual cumulative fee of original single steps and the empty word is $7n + 5$. These equalities compare their respective actually generated programs. They do not combine costs from different ASTs into a single fee.

7.3 Batching Has No Free Pass Either

Proposition 7.4 (Exact accounts for prefixes, batches, and actual calls). Fix the original syntax seed, original substitution σ, and iteration count n. Let

$$\Sigma_n = \sum_{i=0}^{n} r(e[\sigma]^i).$$

The sum of all prefix-trace lengths is Σ_n, and the complete vector-batch program has cost

$$\Sigma_n + 2(n + 1). \tag{14}$$

The trace length of each actual call equals its own call cost; the sum of all call charges equals the sum of all call costs, and the original snapshot is generated from the environments of these actual calls.

Proof. For each i, pairing preserves remaining count node by node, so executing $\widetilde{e[\sigma]^i}$ has trace length $r(e[\sigma]^i)$. Batch syntax adds one original linear embedding node and one accumulation node at each coordinate. Expanding the remaining count inductively over $n + 1$ coordinates gives (14). An actual call frame first reads raw, query, Result, and Trace from its actual occurrence, then uses this raw to generate execution debt; the completed-trace length formula gives the cost equality call by call. The snapshot's old value is the call's actual environment, and its increment is the difference between adjacent environments; coordinatewise expansion identifies it with the original orbit-batch environment. No future call value is supplied as input. For each call j, the whole ledger generates a complete destination for its original mathematical row and an actual same-debt payment for every $k < r(\mathrm{Raw}_j)$; all new exposures are retained in the actual born inventory. ■ The shared pairing and time-action proofs appear in [6], Theorem 9.7 and Supplement S4.3.

The default query executes the original query, the old-value child request, and the actual-successor child request separately; a branch without a source successor uses constant zero. Each

syntax tree has its own node charge and trace. The old inventory, child relation trees, and newly paid entries enter the next inquiry in order. Supplement C.2, “Actual children,” expands every fee constant node by node.

8 The New Ledger Takes Over the Old Ledger—and the Next Request

8.1 The Complete Typed Inventory Must Actually Read Back

A number cannot replace complete material. Let V_P be the physical-language variable type and V_L the lower calculation-language variable type. The original complete material retains both sides' Raw, paid State and Trace, each variable's binding, the complete imported trace, scalar and paired old inventories, and written inventory. Events in the two languages first retain their own types and are then transported by the source binding.

Here Raw retains the environment and original AST; the paid State retains the current pending syntax and its original paid trace, and Trace records every ordered step from the registered syntax to that State. Inventory stores these materials at distinct occurrence positions. The seed operation in (15) adds one seam event and joins two inventories in order.

Theorem 8.1 (Complete typed inventory enters the same admission). For any original frame, source-owned program, and independent lower variable type, the actual query generates a residual request and first paid step; the mother compiler directly compiles that step into responsibility admission. The complete material reads back as the original material itself in the actual target projection. Every physical and lower old event, together with this paid exposure, enters the born inventory occurrence by occurrence, while the original old projections, first whole-ledger write, and canonical successor are jointly preserved.

Proof. First read original raw, environment, and trace from the paid state of the same query, generate the writable request from the boundary in (9), and take the original executor's output on this syntax as the first step. The row obtains initial authority from the actual selection proof in its own fragment; authority at every count is propagated along complete destinations, with no downstream row index. Mother admission takes this actual step as input. The same compilation generates the original first write and every old entry's destination.

Install the complete material as a projection component of the original frame. Admission inherits that projection's entire outcome, yielding equality of types and heterogeneous equality of outcomes. Eliminate the outcome along this type equality and read back the original complete material, not merely a numerically equal result. The physical-language binding $\sigma(x) = \mathrm{const}(\rho_P(x))$ satisfies $\mathrm{ev}_{\rho_L}(e[\sigma]) = \mathrm{ev}_{\rho_P}(e)$ by structural induction on expressions; the imported trace preserves this square step by step. Place the imported physical inventory alongside the complete lower inventory, and the left/right embeddings read back their respective sources event by event. The next mathematical-advance and responsibility-birth branches extend this inventory separately. Original coordinate embeddings are injective in both branches, so old positions do not merge. All old destinations of the first write, the born row, and the current and successor equalities are fields of this admission and hence remain together at the same occurrence. ■ Shared mother admission

and typed transport appear in [6], Theorems 9.5–9.6; the complete old-ledger and restructuring expansions of this paper appear in Theorem 5.6 and Supplements B–C.

A direct reading of occurrence multiplicity. The written inventory H_Q of this query, old written inventory H_o, imported inventory H_m, and physical inventory H_P compose as

$$H_C = \mathrm{seed}(\mathrm{seed}(H_o, H_Q), H_m), \qquad H_B = \mathrm{seed}(H_P, H_C). \tag{15}$$

Position j of the original query trace occupies the born-inventory position

$$\iota(j) = j + |H_o| + |H_P| + 3, \qquad H_B[\iota(j)] = H_Q[j]. \tag{16}$$

The three seed seams explain the final 3; successive left/right readings give the second equality. Cancelling the common offset proves injectivity, and adding the same offset preserves strict order. If the original trace reads the same value twice, those readings still occupy distinct positions. The original completion receipt retrieves this inventory at the exact birth point, and it enters the actual query through the same injection: displaying an inventory tree without passing it into the next inquiry is insufficient.

The complete cochain enters the ledger in the same way. The linear boundary of the original word generates its own coefficient AST, relation words, and paid exposure; updated, old, and every mixed term expand into finite inventory indexed by List positions. The original relation equation actually evaluates in the same environment, and the next binding preserves each coordinate along the original successor. After the new Trace enters typed Born, original disposition, inverse reading, and the whole ledger continue to consume it. A zero sum of cochain entries and the absence of every responsibility are different propositions.

8.2 The Next Environment Is Generated from This Receipt

Proposition 8.2 (Same-receipt total value and actual content update). For native execution at the actual supplied occurrence, let v_{old} be this receipt's old paid value and v_{native} its completed residual value. The source-generated decoder writes

$$v_{\mathrm{new}} = v_{\mathrm{old}} + v_{\mathrm{native}} = \mathrm{ev}_{\rho+\delta}(e). \tag{17}$$

The actual born environment is generated from this same-receipt total. Each birth regenerates its next decoder and old-value baseline from its own receipt, retaining the complete inventory callbacks.

Proof. Native request syntax is original raw minus original paid state. Paid-trace soundness identifies the paid value with the raw value in the original environment, and (10) identifies the request value with the raw effect in the updated environment. The environment-update identity $\mathrm{ev}_{\rho+\delta}(e) = \mathrm{ev}_{\rho}(e) + \mathrm{eff}_{\rho,\delta}(e)$ therefore gives (17). The decoder reads this native value only at the actual settled branch of its supplied occurrence; the next write from the source's own receipt installs the encoded complete total in the actual born environment. Adjacent births each consume their own completed value, and the second baseline reads back from the first actual birth, so the initial environment does not become an eternal baseline. Original scalar/paired inventories and

callbacks are inherited with the same configuration. ■ The shared total value and per-receipt decoder appear in [6], Theorem 8.15 and Supplement S5.2.

The actual paid pair therefore both proves that the material is present and generates the next content. Old plus effect in the active environment of the original history program is its own completed output; actual birth directly installs that output. Two successive births do read two actual program outputs. For the named initial frame of the original history, its active environment is already at stage plus one, and one birth from that frame reads stage plus two. Source Trace, pair Trace, owner, complete four faces, and inverse reading each retain their own indices. When the native value is zero, (17) still writes back the complete old value; it does not refund the new syntax payment.

The difference between two executions must also pay its own bill. Its coefficient program executes at the actual grandnext, with trace length exactly equal to the independent coefficient syntax cost; effect and inverse residual read the same Result. Every new paid exposure first enters written paired inventory, then native and joint inventories; the complete original material, old projections, and next whole-ledger write are retained. The two actual executions and the third difference-word execution are three named occurrences; a shared reading does not turn the third into a free observation.

8.3 Cofinal Sampling Cannot Cut Away Intermediate Payments

Theorem 8.3 (Complete stops, births, and future model of one runtime). Each actual starting point in the original runtime of mathematical frames generates a Type completion receipt whose distance to the original settlement equals the current remaining syntax budget; every intermediate offset retains its actual paid Step. These original receipts generate strictly cofinal stops, with the complete segment between adjacent stops still a tickwise restriction of the same runtime. Complete packet, Raw, environment, State, Trace, and owner are transported to every actual stop. The original complete-word action executes in the actual next inventory and generates the maximal future-observation kernel over all allowed action words and sites.

Proof. Recurse strongly on the current remaining count. A settled branch supplies distance zero and its actual branch equality. In a paid branch, the target is the original next frame and budget decreases by one; obtain the recursive receipt at that exact target and increment its distance by one. Induction over $0 \le h \le B$ gives intermediate budget $B-h$ at offset h; if $h < B$ it is positive, so the actual action must be paid. The terminal settlement has zero budget, and the receipt gives distance $d \le B$, hence $d = B$. Original steps concatenate successively into all prefix traces. The original settlement compilation equality then supplies the next responsibility admission and exact born frame.

The original Stage's admission branch consumes actual admission; its continuation paid branch advances only the mathematical frame; its settled branch passes literal born and the new request to the first admission at or after the current tick. At a starting point t of the original Stage, the admission branch has distance 0 to the first admission at or after the current tick; the continuation branch gives distance $B + 1$ from B actual payments, that settlement, and the subsequent tick.

Let $d(t)$ be this actual distance to the first admission at or after the current tick. The initial node is an admission, hence $d(0) = t_0 = 0$. Define

$$t_0 = d(0), \quad t_{j+1} = t_j + 1 + d(t_j + 1), \quad \ell_j = 1 + d(t_j + 1). \tag{18}$$

The original segment takes tick $t_j + h$ point by point, so $t_j + \ell_j = t_{j+1}$, $t_{j+1} \geq t_j + 1$, and $t_j \geq j$. For any tick, choose the first stop no earlier than it; the complete segment from the preceding stop covers that tick. Each stop's grade, complete packet, and cfg are read back by eliminating the actual admission there; grade does not replace the original tick clock. The original State, every intermediate word action, and paid material are therefore retained in cofinal observation.

Write $\mathcal{W}_j$ for the complete word module at stop j. Its original material gives local paired reading $d_j = (\mathrm{ev}_{\rho_j}, \mathrm{eff}_{\rho_j,\delta_j})$. Pairing node by node and transporting along the next packet's language equality generates the original source action $S_j : \mathcal{W}_j \to \mathcal{W}_{j+1}$; its receiver square reads the old value and effect of the original receiver's own $\rho_j^{\mathrm{receiver}}, \delta_j^{\mathrm{receiver}}$, without identifying that environment with the site port's environment. On the finitely supported direct sum $\mathcal{W} = \bigoplus_j \mathcal{W}_j$, let $\mathrm{pr}_j : \mathcal{W} \to \mathcal{W}_j$ be the site projection and $D_j = d_j \circ \mathrm{pr}_j$ the reader on the whole direct sum. The action S advances the original action by site. Write S_a for letter-by-letter action of a finite word a, with the empty word acting as the identity. The kernel of all allowed words is

$$K_{\mathrm{fut}} = \bigcap_{a \in \{*\}^{<\omega}} \bigcap_j \ker(D_j S_a), \qquad \mathcal{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}. \tag{19}$$

Appending one original action letter proves $S(K_{\mathrm{fut}}) \subseteq K_{\mathrm{fut}}$, so S descends to a model action; quotient classes are equal exactly when all a, j readings are equal. Any currently invisible, action-invariant submodule remains invisible by induction on word length and is therefore contained in K_{fut}, proving maximality. The empty word supplies the canonical restriction to the original current joint completion. The original old words, paid exposures, and relation words of every S_j have entered the same actual next inventory. The maximal kernel uses this actual source action, with no separately supplied future table. ■ The shared Type completion, complete cofinal runtime, and future kernel appear in [6], Theorems 6.12, 9.10, and 9.12, respectively; Supplement D expands this paper's complete inventories and consumers at both ends.

The model now has its complete future fibre, and the inventory has actually reached the next packet. One concrete act remains: the original root must use this model residual to generate its own writable request. The next section acts at that seam.

9 The Original Ledger Speaks: Request, Payment, Answer, and the Next Occurrence

9.1 The Occurrence, the Syntax, and the Original Residual

Let S be any type, let V_s be a family of abelian groups indexed by sorts, and let X_s be the variable types. An environment is a dependent function $\Gamma : \prod_{s\in S}(X_s \to V_s)$. Source expressions consist of variables, constants, addition, unary actions by group homomorphisms, and biadditive actions. Write their evaluation as $[\![a]\!]_\Gamma$ and their source-generated increment as $D_a(\Gamma, \delta)$. The original recursive evaluation law gives

$$[\![a]\!]_{\Gamma+\delta} = [\![a]\!]_\Gamma + D_a(\Gamma, \delta). \tag{20}$$

Here D_a comes from the increment recursion of the original AST. A biadditive node retains the left increment, the right increment, and their joint term; equation (20) does not replace the effect of an arbitrary nonlinear expression by evaluation on δ alone.

Explicitly, $D_x = \delta(x)$, $D_{\mathrm{const}\, v} = 0$, $D_{a+b} = D_a + D_b$, and $D_{f(a)} = f(D_a)$, while a biadditive action satisfies

$$D_{B(a,b)} = B([\![a]\!]_\Gamma, D_b) + B(D_a, [\![b]\!]_\Gamma) + B(D_a, D_b).$$

Induction on the AST proves (20): substitute directly for variables and constants; rearrange the two updates for addition; use the homomorphism law for a unary action; and expand $B([\![a]\!]_\Gamma + D_a, [\![b]\!]_\Gamma + D_b)$ for a biadditive action. Its four terms are exactly the old value and the three increments above.

The syntax charge $c(a) \in \mathbb{N}$ is defined by the original nodes: a variable costs 1 and a constant costs 0; addition and a biadditive action cost the sum of their two subexpression charges plus 1, while a unary action costs its subexpression charge plus 1. A lawful single step $a \rightsquigarrow_\Gamma a'$ binds a variable or executes a prepared original syntax operation, possibly within its context. Every step preserves evaluation and satisfies

$$c(a) = c(a') + 1. \tag{21}$$

A complete execution state is $\sigma = (a', \tau)$, where $\tau : a \rightsquigarrow_\Gamma^* a'$ retains the ordered paid trace starting from the original a. Thus

$$[\![a']\!]_\Gamma = [\![a]\!]_\Gamma, \qquad |\tau| + c(a') = c(a). \tag{22}$$

Equation (21) follows by direct calculation for the four original computation steps. Within an addition, unary-action, or biadditive-action context, the untouched subexpression charges cancel on the two sides, so the charge still drops by 1. Each step preserves evaluation by the definition of binding or of the original operation, and its contexts preserve this equality. Now induct on τ: both identities hold for the empty trace; adjoining a step and applying (21) and evaluation preservation gives (22). The original executor recursively evaluates the left subexpression, then the

right subexpression, then the original operation, appending every step to the execution trace. The length of a complete execution to a constant is therefore exactly $c(a)$.

The completion-receipt type is $\mathrm{Settlement}(\sigma) = \sum_{v \in V_s} (a' = \mathrm{const}\, v)$. From σ, the original executor itself emits a completion receipt or an actual next step; its caller supplies no branch, final value, or target state.

Fix a completely admitted source, root, and exact temporal visit. Each actual occurrence o carries its original complete ledger $L(o)$ and the next complete ledger write E_o. For every old row e, $E_o.\,\mathrm{destination}(e)$ provides both its destination row and that row's lawful evolution. Same-debt identity retains lineage and claim; its budget cannot increase along a same-debt evolution. Payment additionally requires this actual evolution to decrease the budget strictly.

The original residual material belonging to o is

$$m = (\Gamma, \delta, a, \sigma = (p, \tau), e), \tag{23}$$

where $e \in L(o)$ is the actual owning responsibility row. The literal residual and request generated from this same material are

$$r_m = \mathrm{add}(a, \mathrm{linear}(-\mathrm{id}, p)), \qquad Q_m = (\Gamma + \delta, r_m, e). \tag{24}$$

Consequently, for every environment Θ, and for the material's own old environment,

$$[\![r_m]\!]_\Theta = [\![a]\!]_\Theta - [\![p]\!]_\Theta, \qquad [\![r_m]\!]_\Gamma = 0, \qquad c(r_m) = c(a) + c(p) + 2. \tag{25}$$

Complete integer-coefficient words form the free $\mathbb{Z}$-module $F(V, X; s)$. The symbol $[a]$ denotes the complete expression generator with coefficient 1, and ev_Θ is linear evaluation. The material's complete paid boundary is $\beta_m = [a] - [p]$; its integer relation word is generated by τ and has boundary exactly β_m. The syntax expression r_m, the free-module element $[r_m]$, and the relation boundary β_m have their respective types.

The literal pair lift sends a constant v to $(v, 0)$, preserves variables and addition, sends a unary homomorphism f to $(u, du) \mapsto (fu, fdu)$, and sends a biadditive action to

$$((u, du), (v, dv)) \mapsto (B(u, v), B(u, dv) + B(du, v) + B(du, dv)).$$

Induction on expressions therefore shows that the lift, executed in $\mathrm{pairEnv}(\Gamma, \delta)$, gives exactly $([\![a]\!]_\Gamma, D_a(\Gamma, \delta))$. Its linear extension is the original pair lift of complete words.

Theorem 9.1 (The future-model residual belongs to the next original responsibility).

Fix any original source factory $\mathcal{F}$, complete initial frame I, original program cfg, and language identity $\mathsf{cfg.LowVar} = X$. Let P_n be the complete packet generated by this original process at its nth actual cofinal admission point; $n \in \mathbb{N}$ is the admission ordinal, while the actual runtime tick is located recursively by the original stopping receipt. The packet retains its own actual grade, complete frame, source seed, program, and language identity.

Generate the original native receiver from P_n and execute its own registered AST. Let f_n be the frame selected by its original stopping receipt, and let m_n be the complete paid material of f_n. This receipt identifies the actual completion branch of f_n. At that same occurrence, the original responsibility-birth program generates the action G_n. Denote its target by T_n, its original whole-ledger first write by $E_n = T_n.\mathsf{firstSuccessor.ledgerEvolution}$, and its canonical next current by $c_n^+ = T_n.\mathsf{targetAnswerAndNext.nextCurrent}$.

Write $e_n = m_n.e$; debtEntry(m_n) denotes the born row determined by the activation law and initial paid state of the request generated from this material. Then, for every n:

1. The original actual next admission current equals c_n^+ exactly. This current carries complete material m_n^+, retaining the registered environment, registered AST, complete State/Trace, and original owning row of P_{n+1}. It directly generates the writable request $q_n = (m_n^+.\Gamma, m_n^+.a, m_n^+.e)$.
2. If t_n is the dependent transport generated by the original language identity and complete next-packet equality, then

$$[q_n.\mathrm{AST}] = \mathsf{t}_n[r_{m_n}], \qquad b_n = \mathsf{t}_n \beta_{m_n}. \tag{26}$$

   These lie in the same class of the original complete future model. Precisely, for every permitted finite action word $h \in \mathrm{List}(\mathrm{PUnit})$ and every actual native site $j \in \mathbb{N}$,

$$\mathrm{Obs}_j(A_h(\iota_{n+1}[q_n.\mathrm{AST}])) = \mathrm{Obs}_j(A_h(\iota_{n+1} b_n))\,. \tag{27}$$

   Here ι_j is the dependent direct-sum inclusion retaining site and grade, A_h executes the original source action letter by letter, and Obs_j reads the complete old-value/effect pair at site j.
3. The dispositions of the old responsibility and newly born responsibility are, respectively,

$$E_n.\,\mathrm{destination}(\mathrm{oldEntry}(e_n)), \qquad E_n.\,\mathrm{destination}(\mathrm{debtEntry}(m_n)), \tag{28}$$

   The two inputs are first transported along the exact initial-support equality of T_n. Both rows share the same E_n and carry its actual whole-destination coherence evidence.
4. For any result sort R, any dependent consumer that accepts q_n, its future-model class identity, the original compilation result, and the original whole-ledger write obtains R directly from this source-generated object.

Proof. First prove the semantic relation independently of the selected environment. Put

$$Z(V, X; s) = \{w \in F(V, X; s) : \forall \Theta,\ \mathrm{ev}_\Theta(w) = 0\}. \tag{29}$$

This is a $\mathbb{Z}$-submodule. The evaluation definitions for expression addition and the negative homomorphism give

$$[r_m] - ([a] - [p]) \in Z(V, X; s) \tag{30}$$

for any material m. What vanishes in every environment is the difference between these two expressions of the same relation.

Suppose $w \in Z$, and choose any old-value environment Γ and increment δ. The original update-evaluation square gives

$$\mathrm{ev}_{\Gamma+\delta}(w) = \mathrm{ev}_{\Gamma}(w) + \mathrm{eff}_{\Gamma,\delta}(w).$$

The two evaluations are zero, so the effect is zero as well, and the complete updated inventory is $(0,0)$. The literal pair lift evaluates in $\mathrm{pairEnv}(\Gamma,\delta)$ to exactly this complete inventory, so it preserves Z. Every pair environment can be reconstructed from its two coordinates as such a pairEnv; preservation thus remains a statement about all environments. Transport along language equality and complete-packet equality preserves the same relation by equality elimination.

The original action at each site is precisely this literal lift followed by the two transports, so it sends Z into Z at the next site. Induct on the length of h. For the empty word, if j is not the input site, that component of the dependent direct sum is zero; if it is the input site, the complete old-value/effect inventory is $(0,0)$. On adjoining a permitted letter, the original action sends $\iota_k w$ to $\iota_{k+1}\mathsf{a}_k w$, with $\mathsf{a}_k w \in Z$, so the induction hypothesis continues to apply. Every permitted word and every site therefore fails to see the difference in (30). The fibre definition of the original complete future model is simultaneous equality of all these future observations. This gives the asserted model-class identity.

Next identify the actual request. The registered AST of the original birth construction is definitionally r_{m_n}. The original next-packet recursion satisfies

$$P_{n+1} = \mathrm{languageTransport}(\mathrm{sourceBorn}(\mathcal{F}, P_n)). \tag{31}$$

First transport the original residual and boundary into the born packet's language, then identify the next complete packet by (31). This gives (26). The all-environment preservation just proved gives (27).

The same stopping receipt also gives $f_n.\mathsf{action} = \mathsf{settled}$. The original whole-ledger target compiler compiles this actual branch to $\mathsf{debtAdmission}(G_n)$; G_n is the result of executing the original responsibility-birth program at the exact temporal event of the original root. Its exact target-current law identifies c_n^+ with the current of the original source-generated birth. Composing the complete-packet identity (31) with the original actual-current law gives the actual next current $= c_n^+$.

Transport the entire material belonging to P_{n+1} along this current equality to obtain m_n^+. Equality elimination simultaneously preserves the environment, AST, increment, State/Trace, and owning row. The writable request is constructed directly from the corresponding three material fields and is dependently identical to the original request at the next stop; no expression or owning row is selected afresh from a model quotient class.

Finally, both rows in (28) are defined by direct application of the same $E_n.\,\mathrm{destination}$. The original $T_n.\mathsf{firstDestination}$ proves coherence with the complete ledger evolution jointly generated by the original source whole write and the source-generated first step. Package q_n, the model equality,

the original settled-compilation equality, and this whole-ledger evidence into a dependent object; eliminate that object directly to obtain the consumer's result in any R. ■

Equation (30) does not assert that r_m vanishes in every environment. For example, with $a = x, p = 0$, the residual is 0 in the old environment $x = 0$ and 1 in the environment $x = 1$; in both environments, the literal residual still evaluates to the complete boundary. Moreover, q_n is already the next packet's registered AST. A further residual generated from that next packet adds a new negative paid term and increases the charge by at least 2, so it cannot be substituted for q_n.

9.2 The Same Current Delivers the Complete Source Material

For any $\mathcal{F}, I, \mathsf{cfg}$ and language identity of Theorem 9.1, construct a pair reader from the original complete material at the same current. Install it as the calculation reader of the datum at each grade, then apply the original common-query installation to obtain a derived factory $\widehat{\mathcal{F}}$. Record update retains the original source reader, decoding writer, complete inventory callbacks, and source seed; the new datum is installed before the emitter. This section reads the complete pair and supplies the joint consumer at every admission stop generated by $\widehat{\mathcal{F}}$. The general residual-value, payment, and suffix-completion derivations still hold for any original frame and program. The notation P_n, f_n, m_n in this section henceforth refers to the actual runtime of this derived factory.

Let $f = f_n$ be the actual native stopping frame generated by this construction, with $m = \mathrm{paid}(f) = (\Gamma, \delta, a, (p, \tau), e)$. The original material reader generates

$$\mathsf{Raw}_{f,o} = (\mathrm{pairEnv}(\Gamma, \delta), \mathrm{lift}(a)), \qquad u = ([\![a]\!]_\Gamma, D_a(\Gamma, \delta)), \tag{32}$$

and the complete execution trace of this AST in this environment to const u. Write $\epsilon = D_a(\Gamma, \delta)$, $b = \mathrm{born}(f, \mathsf{base})$, and $Q = c(r_m)$.

Theorem 9.2 (Source-generated effect and payment of the original born debt).

Then:

1. The original registered input of b equals Q_m exactly. The next canonical rooted request evaluates to ϵ, also the second coordinate of the complete pair in (32).
2. The original first birth step has already been executed by b, leaving original syntax charge $Q - 1 > 0$. Its original actual action must be paid, giving a same-debt payment π_b. This payment actually lands at the original row destination in the complete whole-ledger successor of b, retaining uniqueness of the same-debt target and satisfying strict debit and no refill.

Proof. The original current-material reader $\mathrm{materialAt}(f, o)$ retains the complete raw, nextRaw, paid State/owning row, old-value trace, pair trace, and relation word. Its pair reader takes from this same complete payload

$$\mathrm{pairEnv}(\mathsf{raw.Env}, \mathsf{nextRaw.Env} - \mathsf{raw.Env}), \qquad \mathrm{lift}(\mathsf{raw.AST}). \tag{33}$$

Constructing $\widehat{\mathcal{F}}$ changes only the calculation reader of the original source factory, and the original $\mathsf{nextCfg}$ inherits it. The common coface, meaning this joint family of source projections, jointly installs the old complete inventory coordinate and the new complete Raw/Result coordinate.

This common installation preserves the original classifier. The complete action reader and the query/result readers read the same Raw restriction, so the original executor generates (32) and its own execution trace.

At the exact occurrence selected by the original stopping receipt, nextRaw comes from that same frame's mathematical settlement branch, with environment $\Gamma + \delta$. The increment in (33) is therefore δ, and the second coordinate of the complete pair is precisely ϵ. This uses the actual branch supplied by the stopping receipt, without substituting nextRaw from another current.

The receipt also gives $p = \operatorname{const} v$. Soundness of the original execution trace gives $v = [\![a]\!]_\Gamma$. Equations (20) and (24) therefore yield

$$\begin{aligned} [\![r_m]\!]_{\Gamma+\delta} &= [\![a]\!]_{\Gamma+\delta} - v \\ &= [\![a]\!]_\Gamma + D_a(\Gamma, \delta) - v \\ &= \epsilon. \end{aligned} \tag{34}$$

The original born AST equals r_m and its registered environment equals $f.\mathsf{activeEnvironment}$. The material's own increment is defined as the active environment minus the registered environment, so

$$\Gamma + \delta = \Gamma + (f.\mathsf{activeEnvironment} - \Gamma) = f.\mathsf{activeEnvironment}. \tag{35}$$

The original born input is thus exactly Q_m. Elimination along the actual next-packet equality and language equality of Theorem 9.1 identifies the evaluation of the canonical rooted request with (34). Across grades, the exact statement uses heterogeneous equality; once the same language and group carrier are recovered, it is ordinary equality.

Now prove payment. The born frame has depth 0, while a depth d in the original frame reads the mathematical current after $d+1$ source steps. Birth has therefore already executed the original $\mathsf{firstStep}$. Induction on the original stepwise source runtime gives its registered-clock identity

$$c(\mathsf{event}_b.\mathsf{state.expression}) = c(b.\mathsf{registered.AST}) - (b.\mathsf{depth} + 1) = Q - 1. \tag{36}$$

By (25), $Q = c(a) + c(p) + 2 \geq 2$, so (36) is positive. If $b.\mathsf{action}$ were mathematically settled, its original settlement charge would be zero, contradicting (36). Eliminating the actual $b.\mathsf{action}$ branch must therefore supply an original paid step. The original payment construction consumes exactly that paid step and the equality identifying it with the actual action.

The payment's debt current uses the original mathematical row. Its same-debt lineage is constructed inductively along the original whole-ledger destination. Every target row with this claim must be the original mathematical debt branch; inherited rows lie in the other claim summand and cannot impersonate it. Hence the same-debt target is unique. The original stepwise receipt identifies the actual next paid State; (21) and the debt component of the actual joint ledger evolution give strict debit. No refill is read from this same row evolution.

The payment must also be connected to the original complete ledger. The payment process's original source restriction and the complete root of the richer query/stock coface each retain their own type; their ledger roots are definitionally the same. The two finite visits recur from the same

initial state with the same successor, so induction on the count identifies the visits. The original registered clock then identifies their complete currents at counts $d+1$ and $d+2$. The ledger evolutions of the two whole-ledger successors are therefore heterogeneously equal, and their target supports are equal. Eliminating support equality and then ledger-evolution identity gives

$$\pi_b.\mathsf{step.evolution} \;\simeq\; (E_b.\,\mathrm{destination}(e_b))\,.\mathsf{evolution}, \tag{37}$$

Here e_b equals the original $\mathsf{currentState.entryAt}(\mathrm{unit})$ of b exactly. This also transports the original stock-root causal-entry authority to the same row. Equation (37) retains the actual strict debit; it does not identify two complete roots with different projection cofaces. ■

A completion event has three outputs of different types: the AST completion value ϵ; the disposition answer of the action G_b^{end} with its original receipt; and the action target's canonical causal answer and next current. The following theorem identifies their common completion occurrence and supplies all three readings to the consumer.

Theorem 9.3 (Original suffix completion, answer, and canonical next).

Use the same source, frame, program, born request, and charge notation as in Theorem 9.2.

1. Generate a remaining material history H_b of length $Q-1$ from the original current of b. Its target equals the original source's completed runtime exactly. The complete completed current, paid State, and execution trace all belong to this same suffix target. The numerical value of the completion receipt is ϵ.
2. The original completed actual action is mathematically settled. At this actual completion event, the original whole-ledger target compiler generates a debt-admission action G_b^{end}. This complete action jointly supplies an answer with its original receipt, canonical causal answer-and-next, complete old-row and born-row destinations, and the actual canonical next current. For any result sort, the original completion consumer directly eliminates this action together with the equality between the completion value and the rooted paid trace evaluation.

Proof. Execute the suffix. Write $\mathcal{R}(k)$ for the original source's complete runtime at count k. Repeated original sealed ticks from its initial runtime give

$$\mathcal{R}(k).\,\mathrm{advance}(\ell) = \mathcal{R}(k+\ell), \qquad c(\mathsf{event}(k).\mathsf{state.expression}) = Q-k. \tag{38}$$

The born current is $\mathcal{R}(1)$. Generate H_b tick by tick for $Q-1$ steps from here; then

$$H_b.\mathsf{target} = \mathcal{R}(1).\,\mathrm{advance}(Q-1) = \mathcal{R}(Q). \tag{39}$$

The original completed frame changes only the depth of this same born frame to $Q-1$, retaining the original root, registered input, packet program, environment reader, and inventory. The original count/current law identifies its complete current with the current in (39), and its paid State equals that of the original completed event. This identifies the complete current, not just a final value. H_b continues from the existing born current rather than restarting the initial execution.

Equation (38) at $k=Q$ gives charge 0. The original completion construction gives a constant endpoint. An actual paid action would require a natural-number budget below 0, so the actual

action must be mathematically settled. The completion-value law of the same State/Trace, together with born-input identity, gives completion value $[\![r_m]\!]_{\Gamma+\delta} = \epsilon$. The complete H_b and the already-paid first birth step jointly pay for the entire original AST, with trace length exactly Q.

The rooted material of Theorem 9.1 is the complete transport of this born registered input to the actual next current. Its paid State also satisfies (22), so that State evaluates in its own environment to the same ϵ. This gives heterogeneous equality of the completion value and the rooted paid State evaluation; it does not identify the State after the first birth step with the complete final State.

Finally, execute the original whole-ledger target birth program at this completion root's exact temporal event. The original compiler directly uses the actual settled branch just obtained to return $\mathsf{debtAdmission}(G_b^{\mathrm{end}})$. Its target retains the original completion event, complete source successor, original first step, complete old projection, and complete first write. The two completion rows read

$$E_b^{\mathrm{end}}.\,\mathrm{destination}(T_b^{\mathrm{end}}.\mathsf{translatedSourceEntry}), \qquad E_b^{\mathrm{end}}.\,\mathrm{destination}(T_b^{\mathrm{end}}.\mathsf{bornEntry}). \tag{40}$$

The action's answer $G_b^{\mathrm{end}}.\mathsf{answer}$ carries $T_b^{\mathrm{end}}.\mathsf{Receipt}(G_b^{\mathrm{end}}.\mathsf{answer})$, while the target's canonical causal answer-and-next supplies the original $\mathsf{targetAnswerAndNext.answer}$ and $\mathsf{nextCurrent}$. These are different dependent readings of the same action. The original exact target-current law identifies the actual current of $\mathrm{nextBorn}(\mathrm{completedBorn}(b))$; the original successor-validity law proves that this is the inquiry's lawful successor and preserves the generated living law.

The original completion consumer first obtains the complete action, then directly receives its canonical causal answer, action receipt, completion-value relation, rooted trace/effect relation, original compilation equality, next-current equality, successor validity, and whole-destination evidence. Since the action itself is passed to the consumer, both the old row and the born row in (40) can be read directly. No separate answer, receipt, target ledger, or next current is appended. ■

9.3 Zero Effect Still Owes a Receipt; New Debt Cannot Refill Old Debt

Take $a = \mathrm{const}\, v$ and the paid endpoint $p = \mathrm{const}\, v$ in the same source material. Under every actual environment increment, $D_a(\Gamma, \delta) = 0$, yet the original next request remains

$$r = \mathrm{add}\big(\mathrm{const}\, v, \mathrm{linear}(-\mathrm{id}, \mathrm{const}\, v)\big), \qquad c(r) = 2.$$

The original first step executes the negative-homomorphism node, leaving charge 1. The next actual born payment executes the addition node, strictly paying $1 \to 0$ and producing the complete $\mathrm{const}\, 0$ endpoint. Zero effect erases neither these two actual syntax operations nor their sources or payment lineage.

The completion disposition can also generate the next request from the same occurrence. In this example, that new request is literally $r - \mathrm{const}\, 0$, with charge 4. Its fresh admission pays for its own first step and registers its own debt. Both responsibilities and claims in the extended ledger use sum types: the old ledger belongs to inl and the newly born debt to inr. The original initial-origin

construction directly eliminates these two branches for every complete initial row, obtaining either its old-row origin or its unique born-row origin without an external old/fresh classification field.

The old zero-budget row retains same-debt continuation under the original whole-ledger write; the original completion interface even supplies a witness that its successor same-debt target is nonempty. The new debt's positive syntax budget belongs to another source-generated fresh row and cannot be exchanged with the old debt's budget. Same-debt no refill constrains continuation of the old lineage/claim, while allowing the occurrence lawfully to generate a new responsibility with its own claim, first step, and receipt. The completion receipt settles the original AST here; final same-debt settlement still requires both the original terminal receipt and an empty successor same-debt fibre.

The original ledger can now identify the complete future residual, make an actual payment, complete the original input, deliver an answer, and generate the next lawful request from the complete old ledger.

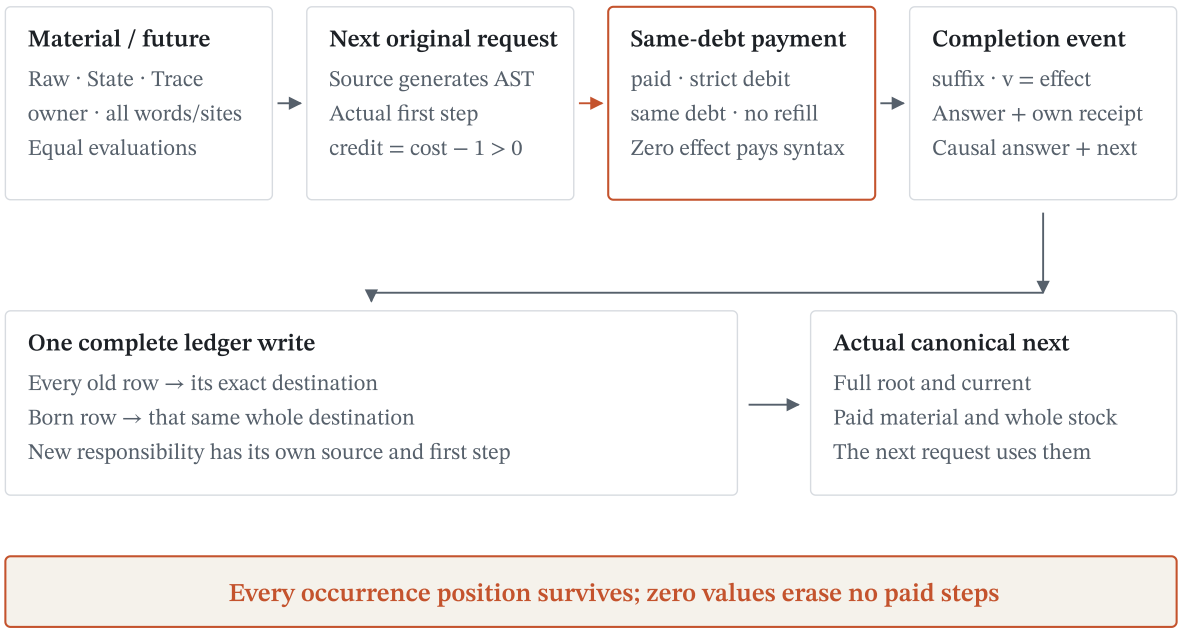


Figure 3 The ledger can pay for itself—and turn its own page. The literal residual of the original material and its paid boundary agree across all environments, permitted action words, and sites; the writable request is generated from the complete material. Actual syntax payment and suffix completion each consume their original steps. The same completion action delivers the numerical completion relation, an answer with its receipt, old/new rows, and the canonical successor. Arrows denote actual generation or transport along exact current equality; distances between boxes have no numerical meaning.

10 One Real Payment. No Amount of Moving Can Refill It.

10.1 Track the Same Debt Through the Original Process

Definition 10.1 (Currents and microsteps for a fixed original debt). Fix a source-generated process P, with state type S_P and actual successor function $\sigma_P : S_P \to S_P$. Each state $s : S_P$ determines a complete root, registered visit, and the support $u_P(s)$ emitted at that visit; write its complete live ledger as $\mathcal{L}_P(s) := \mathcal{L}(u_P(s))$. Fix an original debt a. Its same-debt current type and budget are

$$X_a := \sum_{s:S_P} \sum_{e:\mathcal{L}_P(s)} (a \sim_{\text{debt}} e), \qquad B(s, e, -) := b(e). \tag{41}$$

A microstep from $x = (s, e, -)$ consists of a target entry f in the complete ledger of the actual successor $\sigma_P(s)$, a row evolution from e to f, and a proof that every entry in that successor having the same debt as e equals f. The state of the new current x' is therefore $\sigma_P(s)$. Same-debt identity is obtained by composing the original witness with the row evolution's same-debt identity, and Proposition 5.2 gives $B(x') \le B(x)$.

This single-debt interface requires a unique same-debt target in the successor. Choosing either of two split children does not finish tracking the parent debt. A split must first satisfy the complete coverage contract of Section 5; entry into this section's single-debt path additionally requires the unique-target witness above.

A finite non-refilling history is an identity history, one microstep, or a history with one more microstep appended. An identity history adds no time step and represents no progress. Induction on history length shows that its final budget is no larger than its initial budget.

Definition 10.2 (Payment and a paid macrostep). A payment is such a microstep together with its actual strict-payment fact $B(x') < B(x)$. A paid macrostep consists of a finite history, one payment, and another finite history:

$$x \rightsquigarrow^* u \xrightarrow{\text{pay}} u' \rightsquigarrow^* y. \tag{42}$$

Either surrounding history may be empty and may carry, change presentation, or transfer the responsibility. Every step passes through the fixed process's actual successor and retains same-debt identity. The macrostep constructor accepts no separate field saying that the final budget is smaller than the initial budget: this inequality must be derived from the actual payment in the middle.

Main Theorem 10.3 (Paid macrosteps are well-founded without a total closure law). For any fixed P, a, let $y \prec_a x$ mean that a paid macrostep exists from x to y. Then $\prec_a$ is well-founded. In particular, a paid macrostep cannot return to the same complete debt current, and a zero-budget current has no paid macrostep. The theorem assumes neither a total emission law nor a supplied final state or complete future history.

Proof. Well-foundedness needs no total law. The surrounding histories can only avoid increasing the budget; the actual payment between them supplies the decrease, and the natural-number budget completes the argument. Induct on the two finite histories in (42) to obtain $B(u) \le B(x)$ and $B(y) \le B(u')$. The payment supplies the strict inequality in the middle, yielding

$$B(y) \le B(u') < B(u) \le B(x), \quad \text{hence } B(y) < B(x). \tag{43}$$

Thus $\prec_a$ is a subrelation of the strict natural-number order pulled back along $B : X_a \to \mathbb{N}$. The strict order on natural numbers is well-founded, its inverse image is well-founded, and a subrelation remains well-founded. If a macrostep returned to its original current, (43) would give $B(x) < B(x)$; if $B(x) = 0$, it would give $B(y) < 0$. Both are contradictions. This well-foundedness is determined by the already-generated paid macrosteps and the budget map. ■[3]

Direct Refutation 10.4 (A designated equal-budget microstep cannot be payment). Fix an actual microstep $e : u \to u'$ and an equality $B(u') = B(u)$. Certifying this designated e as the payment in Definition 10.2 would require $B(u') < B(u)$, contradicting the equality. Appending any finite histories before and after it does not change this microstep's own endpoints or their budget equality. If either surrounding history contains another actual strict payment, that payment can instead serve as the payment of a macrostep. The refutation always excludes certification of e itself as payment. The budget equality for an unchanged carry and the equal-budget microstep have additional formal counterexamples (see Code and Data Availability). The $0 \to 0$ transfer in the original arithmetic whole ledger of Section 13 supplies a concrete nonpayment instance.

Well-foundedness concerns the actual paid macrosteps of this fixed original debt, not a demand that all motion stop. The world may continue emitting writes, fresh responsibilities may be lawfully generated, and the same responsibility may remain at a position where payment or settlement evidence has not been obtained. The next section supplies the indispensable contract connecting "payments cannot continue forever" to "settlement actually occurs."

11 Two Keys to the Settlement Door

11.1 The Original Terminal Receipt Must Survive the Next Step

Definition 11.1 (Local termination and final settlement). For $x = (s, e, -) : X_a$, a local termination $\mathrm{LocalTerm}(x)$ is a faithful terminal occurrence emitted by its exact current root. It carries a proof that structural compilation is terminal, the complete ledger's entrywise terminal evolution, and proofs identifying these objects with the original whole-ledger compilation. It also inherits the ordinary-outgoing-event exhaustion condition of complete event admission. Its row receipt is obtained by applying this complete terminal evolution to the original entry e.

Define the same-debt fibre in the actual successor and the settlement type by

$$T(x) := \sum_{f : \mathcal{L}_P(\sigma_P(s))} (e \sim_{\mathrm{debt}} f), \qquad \mathrm{Settle}(x) := \mathrm{LocalTerm}(x) \times \mathrm{IsEmpty}(T(x)). \tag{44}$$

Here $s : S_P$, and $\sigma_P(s)$ is the successor actually generated by the original source process, including its source-fixed local terminal handoff. After receiving a terminal receipt, the consumer cannot substitute a different debt-free world for inspection.

[3]The shared settlement construction is Theorem 6.7 of [6].

Proposition 11.2 (A terminal receipt does not automatically become settlement). A settlement value projects to the exact terminal receipt of the original ledger entry. Conversely, even with local termination available, the final settlement type is empty whenever the actual successor contains a same-debt entry.

Proof. In the forward direction, take $\mathrm{LocalTerm}(x)$ and apply its complete terminal evolution to e. Conversely, given $t : T(x)$, the second field of any proposed settlement eliminates t to a contradiction. A target with unchanged lineage and claim cannot be counted as fresh debt merely because its carrier differs. Equation (6) defines precisely this check across carriers. ■

The check does not require the actual successor to be blank. It requires the original debt to be absent. After target six is completed, the original arithmetic ledger and its actions continue; what is excluded is exactly the settled action debt. Whole-ledger accounting ends a responsibility, not the world.

11.2 Well-Foundedness Still Needs a Total Law

Definition 11.3 (Total emission closure law). For fixed P, a, a total emission closure law is an actually supplied function

$$\mathrm{emit} : \prod_{x : X_a} (\mathrm{Settle}(x) \sqcup \mathrm{PaidMacro}(x)). \tag{45}$$

For each current in this type, it supplies an exact settlement or a completed finite macrostep containing actual payment. A hope that some successor might exist somewhere, or that continuation ought to occur whenever termination does not, is not (45).

Main Theorem 11.4 (A total law generates a finite settlement history). Given the total law of Definition 11.3, every $x : X_a$ has a finite generated history whose branches agree with the law's actual outputs. Its endpoint carries a settlement value of (44), jointly supplying the original compiler's entrywise terminal receipt and a proof that the exact successor's same-debt fibre is empty.

Proof. Well-foundedness only ensures that payments cannot continue indefinitely. The total law must supply the branch taken at each step, and the history must match those outputs step by step. First define a dependent inductive type $H(x)$. Its terminal constructor receives $q : \mathrm{Settle}(x)$ and $\mathrm{emit}(x) = \mathrm{inl}(q)$. Its continuation constructor receives $m : \mathrm{PaidMacro}(x)$, $\mathrm{emit}(x) = \mathrm{inr}(m)$, and $H(m.\,\mathrm{target})$. A history therefore cannot choose a different decreasing branch partway through.

Recurse along the well-founded relation of Main Theorem 10.3. At x, compute $\mathrm{emit}(x)$: for the left branch, immediately construct $H(x)$ from that output and its equality; for the right branch, the macrostep itself supplies $m.\,\mathrm{target} \prec_a x$. Recurse only at this exact target, then append the current output and its equality. Well-founded recursion ensures that the result is a finite inductive object. Follow its continuation constructors to the tail to obtain the dependent pair $\sum_y \mathrm{Settle}(y)$, then take the receipt of the original terminal evolution at the exact ledger entry of y. Each recursion node uses one actual output of the total law; well-foundedness ensures termination along those outputs. ■

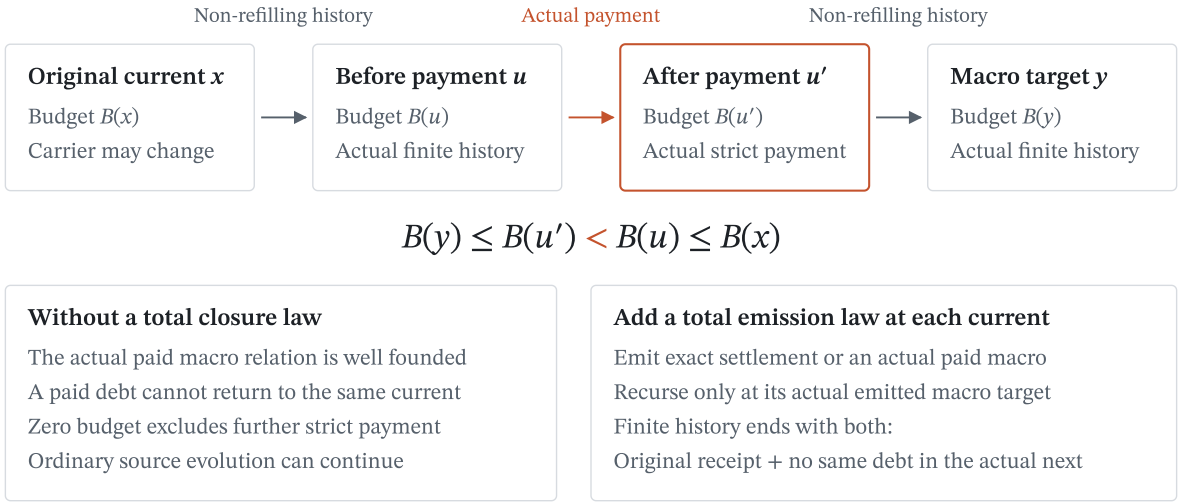


Figure 4 The two checks for same-debt macrosteps and settlement. The original debt's lineage and claim are fixed throughout. Neither finite surrounding history can refill its budget; one actual payment supplies strict decrease. The total emission law supplies "settlement or payment" at each current. Final settlement additionally requires the original terminal receipt and no same debt in the actual successor. Zero budget excludes the payment branch without generating settlement evidence.

11.3 An Exhausted Budget Is Not a Miracle

Corollary 11.5 (The exact zero-budget distinction). If $B(x) = 0$ and a total emission closure law is supplied, it must emit settlement at x. If $B(x) = 0$ and the local terminal type is empty, no such total law exists for this debt.

Proof. Expand (45). The payment branch contradicts the zero-budget conclusion of Main Theorem 10.3, so a total law can only leave its actual settlement output. If local termination is also empty, the first field of the settlement branch gives a contradiction as well; the total law itself therefore cannot exist. ■

Counterexample 11.6 (Zero budget at the original fluid-medium current face). Fix any parameters and stage permitted by the original fluid-medium source. Its operational responsibility row has zero budget, while its vocabulary's faithful terminal-payload type is empty. The original source still has its actual structural continuation. Zero budget has neither removed these occurrences nor generated termination. For this original row, paid macrosteps, settlement, and total emission closure laws are all empty: the first cannot strictly decrease below zero; the other two are eliminated by the terminal payload and the total law's two branches, respectively. This is a type exclusion at the original current face.

Example 11.7 (Stage-three payment with an actual call to the total law). The original third-stage occurrence-payment construction accepts an actual stage-payment receipt. This input fixes its pending responsibility and exact installed target. The pending budget is 1 and the installed budget is 0; the receipt compiles to a unique same-debt target and its strict step in the actual successor ledger. The installed current emits its original termination and then enters a successor where the debt is inactive. The target's old-claim branch cannot equal the original debt-claim branch, showing that $T(x)$ is empty. The construction exhausts its same-debt currents: at the pending current, it emits a paid macrostep built from the original step; at the paid current, it

emits the settlement just constructed; an inactive current cannot inhabit the same-debt type. This gives a concrete total law of (45), to which Main Theorem 11.4 applies to generate the finite history and terminal receipt. The actual payment input is precisely that stage-payment receipt; the total law does not solve its underlying domain conditions.

Four objects easily conflated as "completion" are now distinguished: a whole-ledger answer, strict payment, local termination, and final settlement. The first prevents omission of ledger entries; the second ensures actual same-debt budget decrease; the third supplies the original source's terminal receipt; the fourth also checks the next step. Actual constructions can connect them, but they cannot impersonate one another.

12 Change the Language. Bring the Old Ledger.

12.1 New Expression Is Not Debt Amnesty

When expressive capacity fails, the original process can generate a minimal extension retaining the old structure and realize it as a typed world revision [6] (Theorems 8.4 and 8.6). Such a revision must still answer: once what the old language could not express appears, do the old responsibilities fall through the seam?

Definition 12.1 (Expressive revision carrying the original ledger). A revision starts from the old root's exact causal responsibility, actual obstruction, and original successor. The expressive obstruction here carries proofs that both the demand's expression fibre and its lawful-realization fibre in the old language are empty. The revision supplies a new world, new source-generated root, and its first write; the new initial support reflects back to that original successor support. The entire old target live ledger enters the new initial ledger by an embedding i with a left inverse. Old claims commute with translation, and inherited budgets do not increase. The new ledger also contains an obstruction-generated entry η whose responsibility, occurrence incidence, and claim have no preimage in the old coordinates. Its budget is zero, and its claim holds at the exact new support.

The crucial condition is stronger than a set embedding. Each $i(e)$ and η carries an actual generated row in the new root's original first whole-ledger write, with proof that the row commutes with that write. The presentations of the old event family and new initial event family also send the original emitted event to the new original emitted event. The new root is generated together with its terminal-handoff law. A revision cannot be dressed up first and then left to its user to select a later world.

Proposition 12.2 (The minimal extension retains the complete old ledger). The obstruction-generated minimal responsibility carrier is $\mathcal{L}_{\mathrm{old}} \sqcup \{\eta\}$, with old budgets read along the original embedding and $b(\eta) = 0$. Every admissible revision satisfying Definition 12.1 receives a canonical faithful factorization of this minimal extension. It retains the old ledger and sends the new direction attached to the original obstruction exactly to the η used by the first write. This factorization is unique among the specified preserving morphisms of responsibility carriers, occurrence incidences, and claims.

Proof. The old entries have only one place to go, and so does the new direction. This gives both existence and uniqueness. On the old-ledger branch, preservation forces the map to be i; on the new-direction branch, preservation of the obstruction coordinate forces it to be η. Defining the map branch by branch gives existence. Any other preserving map agrees on both branches, so function extensionality gives uniqueness. The left inverse of i prevents identification of old entries, while the absence of an old preimage prevents confusion of the new entry with an old one. The attachment equality of the generated direction then identifies the minimal extension's obstruction branch with the original new row. The generated-row fields of Definition 12.1 ensure that this direction actually enters the first whole-ledger write. Responsibilities, occurrence incidences, and claims are handled separately by the same construction; this initiality does not claim to determine arbitrary additional domain structure. ■[4]

12.2 Richer Does Not Mean Free to Assemble

A source-fixed revision has two admission shapes. The exact minimal shape requires the new network, translation, support, old-ledger embedding, new entry, fact, and semantic change to equal the canonical obstruction-generated objects. The richer shape allows a larger representation, but requires the new root's law surface still to be generated by its new world semantics, and its first whole-ledger compiler to **actually select** a genuine split: two distinct target entries have the same origin, precisely the transported entry of the original causal responsibility. They use the same complete split coverage; a compatible but unselected receipt cannot be attached instead. Every initial ledger entry must belong to the inherited old ledger or η, and each target's origin must also lie in one of these two families.

The second shape does not reduce every revision to the minimal network, but it admits no extra coordinates without origins. By Section 5, each child budget is strictly below the original transported parent budget. If the concrete action also requires separation of bearers, their inequality is read from the receipt under the same source law. The actual first write, minimal factorization, exact split selection, and two strict debits are thus combined in one dependent object. Conversely, this two-child action type is empty if the exact target ledger has at most one entry or if the original compiler's exact split-coverage type is empty: the first contradicts distinct children, and the second directly eliminates the required coverage.

A revision object carrying all these components is initially an admissible realization. To be the actual revision of an inquiry, it must still be emitted by that inquiry's fixed source program. Admissibility and actual emission occupy different positions in the type, preventing “there exists a writable new language” from impersonating “the original process has reached it.”

Theorem 12.3 (Old and new responsibilities enter the same actual disposition). Given the grounded revision above, actually emitted by the fixed inquiry program, the generated causal authority of every inherited old entry $i(e)$ and new entry η passes along the same first write to its exact successor. Each then receives the original revised root's canonical entrywise answer. All these answers have the same next current, equal to the next current actually generated by that root.

[4]Shared initiality and attachment of the generated direction are Theorems 8.4 and 8.6 of [6].

Proof. The old and new entries have different origins and one meeting point: that first write. For each $i(e)$, use its generated row in the first write to construct initial causal authority; for η, do the same with its own generated row. Take the successor equality of the first write once. Apply the same exact continuation operation to all these authorities, obtaining their respective successor causal authorities. Then apply Theorem 3.5: their ledger-entry answers may differ, but every successor current is read from the same occurrence at this exact revised root and exact visit. Transitivity of the equalities identifying both readings with this original successor gives equality of all next currents. The old responsibility is not left behind at the translation, and the new responsibility is not stranded beside a semantic-change certificate. ■[5]

Example 12.4 (An already-constructed two-tick revision). The concrete construction of Proposition 8.9 in [6] starts from an old responsibility with budget 2; both original expression and realization fibres are empty. Its revised initial ledger contains both the inherited old entry and a new entry with budget 0. The new root's currents are natural numbers and its original successor adds one. At the first write $0 \to 1$, the entries undergo transfers $2 \to 2$ and $0 \to 0$, respectively. At current 1, both canonical answers continue to current 2; the next inquiry tick then advances $2 \to 3$ through this generated root. Both entries obtain disposition and succession from this actual write. Each transfer preserves its budget, and the new entry continues as a live zero-budget responsibility. ■[6]

New expression therefore does not absolve the responsibilities it creates. The old ledger cannot leave the stage, and the new ledger cannot hang on the wall as decoration: both enter the same occurrence's disposition. A theory can become more eloquent without the ledger losing a single entry.

13 The Whole Ledger Also Produces Mathematical Consequences

13.1 A Prime-Three Coordinate Born from the Original Transfer Receipt

The target-six debt really was settled. Another original arithmetic chain has not settled its ledger, yet its original action has already left a mathematical consequence that a particular reader cannot erase.

At the first stage, the original unit-arithmetic root's current cardinality shadow is 2, and its complete factor inventory does not yet contain the prime 3. The original unit action advances the target to the next current. Its transfer receipt supplies the target's complete factor inventory and generates the exact prime-three key. Reading the nonadvancing zero-action target instead still yields no such key. The reader construction takes the key from this already-occurred unit transfer; neither the key nor a nonzero proof is an input.

> **Construction: two coordinate layers, prime action, and endpoints.** Fix the unit-arithmetic source above. The unit history h_n written at stage n has cardinality shadow $n + 2$; its dependent factorial fold has cardinality $(n + 2)!$. Write the actual

[5] This corresponds to Theorem 8.7 of [6].
[6] The concrete two-tick construction and proof of Proposition 8.9 in [6].

prime-factor set as $\mathcal{P}_n$, with multiplicity $m_n(p) = v_p((n+2)!)$ for p, retaining every positive prime-power occurrence:

$$\mathcal{R}_n := \{(p,k) : p \in \mathcal{P}_n,\ 1 \le k \le m_n(p)\}.$$

The **inner coordinates** are (q, j, ϵ), with $q \in \mathcal{P}_n$, $0 \le j \le m_n(q)$, and $\epsilon \in \{0,1\}$. The integer module K_n consists of arrays satisfying all the recurrences

$$v(q, j+1, \epsilon) = q\, v(q, j, \epsilon), \qquad 0 \le j < m_n(q).$$

Reading $j = 0$ gives $K_n \cong \mathbb{Z}^{\mathcal{P}_n \times \{0,1\}}$; the inverse sends a base coordinate $a(q,\epsilon)$ to $q^j a(q,\epsilon)$. Induction on j verifies that both composites are identities. The dual reversal τ exchanges $\epsilon = 0, 1$, while the prime action multiplies the inner q coordinate by q.

The **outer coordinates** retain every factor row $r = (p,k) \in \mathcal{R}_n$. The complete relation module and its action are

$$M_n := K_n^{\mathcal{R}_n}, \qquad (E_n v)(r; q, j, \epsilon) := q\, v(r; q, j, \epsilon).$$

The outer p specifies the factor relation being inspected, while the inner q specifies the prime coordinate of that relation's value. They are independent in the complete module.

Endpoints and boundaries come from the vertex module $K_n^{\{*\} \sqcup \mathcal{R}_n}$. The coordinate $*$ denotes the whole position; the remaining positions denote the quotient entries of the factor rows. The factorization differential is

$$(d_n w)(p,k) = (I - \tau) w(*) - p^k (I - \tau) w(p,k).$$

Put $\iota_n(a_0, a_1)(q, j, \epsilon) = q^j a_\epsilon$. The left and right endpoints take $\iota_n(1,0)$ and $\iota_n(0,1)$, respectively, only at the whole position, with all quotient entries zero. The left endpoint's differential is β_n and the right endpoint's is $-\beta_n$, where

$$\beta_n(r; q, j, 0) = q^j, \qquad \beta_n(r; q, j, 1) = -q^j.$$

These coordinates are generated with the unit history. Successor restriction forgets only the newly added primes, powers, and factor rows, retaining the recurrences, action, and endpoints above.

Take the action cokernel and endpoint-boundary class of this complete relation module:

$$C_n := M_n / \operatorname{im}(I - E_n), \qquad z_n := [\beta_n] \in C_n. \tag{46}$$

Here “cokernel” means treating entries of the form $(I - E_n)v$ as zero. For an actual factor row $r = (p,k)$, take the integer reading $\ell_r(v) := v(r; p, 0, 0)$ at the same-prime, zero-exponent, left-dual coordinate. The construction formulas immediately give

$$\ell_r(\beta_n) = 1, \qquad \ell_r((I - E_n)v) = (1-p)\ell_r(v). \tag{47}$$

Equation (47) is therefore a coordinate calculation on the complete integer module: it deletes no other coordinates and applies no prior complexification.

Theorem 13.1 (The original receipt generates a nonzero reading, retained at every successor). The prime-three row generated by the original first-stage unit transfer gives $z_1 \neq 0$. The arithmetic reader of the original symbolic block action sends its endpoint cokernel class exactly to z_1 and is therefore nonzero. The original stage restrictions retain the endpoint class, so $z_{k+1} \neq 0$ for every natural number k, with each corresponding original block reading nonzero. Each stage retains the original emitted event, complete ledger writeback, and actual successor.

Proof. An integer coordinate and a parity check block zero. Substitute $p = 3$ for the row generated by the original receipt. If $z_1 = 0$, equation (46) supplies an element v of the integer module with $\beta_1 = (I - E_1)v$. Applying ℓ_r and then (47) gives $1 = -2\ell_r(v)$, contradicting integer parity.

The base coordinates of the symbolic relation module are $\mathcal{A}^{\mathcal{R}_n \times \mathcal{P}_n \times \{0,1\}}$, where $\mathcal{A} = \mathbb{Z}[\alpha_p, \gamma_p \mid p \text{ is prime}]$; its inner q dual block acts by $\left(\begin{smallmatrix} \alpha_q & \gamma_q \\ \gamma_q & \alpha_q \end{smallmatrix}\right)$. Let $\mathrm{ev} : \mathcal{A} \to \mathbb{Z}$ send α_p to p and γ_p to zero. The arithmetic reader is explicitly

$$A_n(v)(r; q, j, \epsilon) = q^j \,\mathrm{ev}(v(r; q, \epsilon)).$$

A coordinatewise calculation gives $A_n(I - E_n^{\mathrm{blk}}) = (I - E_n)A_n$, so the symbolic operator's image is sent into the integer operator's image, inducing a semilinear reader on cokernels. The symbolic boundary has value $(1, -1)$ at every inner dual coordinate; A_n sends it exactly to β_n in the construction. The block reader consequently reads the nonzero cokernel class established above.

Write the original restriction as $\rho_n : C_{n+1} \to C_n$, with exact endpoint equality $\rho_n(z_{n+1}) = z_n$. Induct from $z_1 \neq 0$. If the successor class were zero, linearity would send it to zero, and the endpoint equality would force the previous class to be zero, a contradiction. The commuting identities of the original installed reader retain the ledger and successor equalities at every stage. ■[7]

The extended reader adds a dependent projection face while retaining the original structural source, complete event admission, law surface, emitter, and whole-ledger compiler. Its projection law is fixed; the transfer receipt, target, and arithmetic class in its payload all come from the same original event. In particular, the original ledger entry still undergoes an actual $0 \to 0$ budget transfer and has not been reclassified as strict payment. A new mathematical consequence and settlement of an old responsibility are distinct capabilities: whole-ledger accounting allows the former to occur while rejecting its impersonation of the latter.

Lemma 13.2 (Determinant annihilation at a finite stage). At every finite stage n above, arithmetic specialization of the source's prime–dual block determinant gives

$$D_n = \prod_{p \in \mathcal{P}_n} (1-p)^2 \neq 0, \qquad D_n z_n = 0. \tag{48}$$

[7] Nonzero from stage 1 onward; no identical assertion is made at stage 0.

Consequently, $1 \otimes z_n = 0$ in $\mathbb{C} \otimes_{\mathbb{Z}} C_n$. The product takes one two-dimensional block for each actual prime. It is the determinant on the inner base coordinates, not a determinant of the full matrix on M_n counting the outer factor rows repeatedly.

Proof. The task is to construct a nonzero integer annihilating the endpoint class without division. Take adjugate cofactors at the symbolic level, then specialize arithmetically. In the symbolic coefficient ring $\mathcal{A} = \mathbb{Z}[\alpha_p, \gamma_p \mid p \text{ is prime}]$, the source's inner two-dimensional action block is $\left(\begin{smallmatrix} \alpha_p & \gamma_p \\ \gamma_p & \alpha_p \end{smallmatrix}\right)$. The inner block of $I - E_n^{\mathrm{blk}}$ and the determinant of the entire inner action are therefore

$$T_p = \begin{pmatrix} 1-\alpha_p & -\gamma_p \\ -\gamma_p & 1-\alpha_p \end{pmatrix}, \qquad \Delta_n = \prod_{p \in \mathcal{P}_n} \delta_p, \qquad \delta_p = (1-\alpha_p)^2 - \gamma_p^2.$$

At every outer row and inner prime, the symbolic boundary has dual value $(1, -1)$. Its eigenfactor under T_p is $1 - \alpha_p + \gamma_p$. Take the division-free adjugate cofactor

$$a_{n,p} := (1 - \alpha_p - \gamma_p) \prod_{q \in \mathcal{P}_n \setminus \{p\}} \delta_q.$$

Factor $\delta_p = (1 - \alpha_p + \gamma_p)(1 - \alpha_p - \gamma_p)$ to obtain $(1 - \alpha_p + \gamma_p)a_{n,p} = \Delta_n$. At each outer row, assigning the inner p dual coordinate the value $a_{n,p}(1, -1)$ constructs a preimage w_n^{blk} in the symbolic relation module, satisfying coordinatewise

$$(I - E_n^{\mathrm{blk}}) w_n^{\mathrm{blk}} = \Delta_n \beta_n^{\mathrm{blk}}.$$

Thus the symbolic boundary class multiplied by Δ_n is zero in the action cokernel. Arithmetic specialization $\alpha_p \mapsto p,\ \gamma_p \mapsto 0$, followed by the extension of the inner base coordinates with q^j from the construction, is exactly the preceding semilinear reader. It sends the preimage equality to $(I - E_n)w_n = D_n \beta_n$. Every actual prime satisfies $p \neq 1$, hence $D_n \neq 0$, giving (48).

Finally, tensor balancing gives $D_n \otimes z_n = 1 \otimes D_n z_n = 0$. Multiply the complex coefficient by D_n^{-1} to obtain $1 \otimes z_n = 0$. This elimination is performed separately at every finite stage, with possibly different D_n at different stages. ■

13.2 Every Complex Reading Says Zero. The Integer Class Remains.

Theorem 13.3 (The original integer consumer does not factor through the complete complex observation family). Form the global compatible integer cokernel using the original restrictions:

$$G := \{(g_n)_{n \in \mathbb{N}} \in \prod_n C_n : \rho_n(g_{n+1}) = g_n \ \text{ for every } n\},$$

Let $\pi_n : G \to C_n$ be the nth projection. Compatibility of the endpoint classes gives $z = (z_n)_n \in G$. Define all finite-stage complexification observations and the original first-stage integer consumer by

$$F(g) := (1 \otimes \pi_n(g))_{n\in\mathbb{N}} \in \prod_n (\mathbb{C} \otimes_{\mathbb{Z}} C_n), \qquad J(g) := \pi_1(g). \tag{49}$$

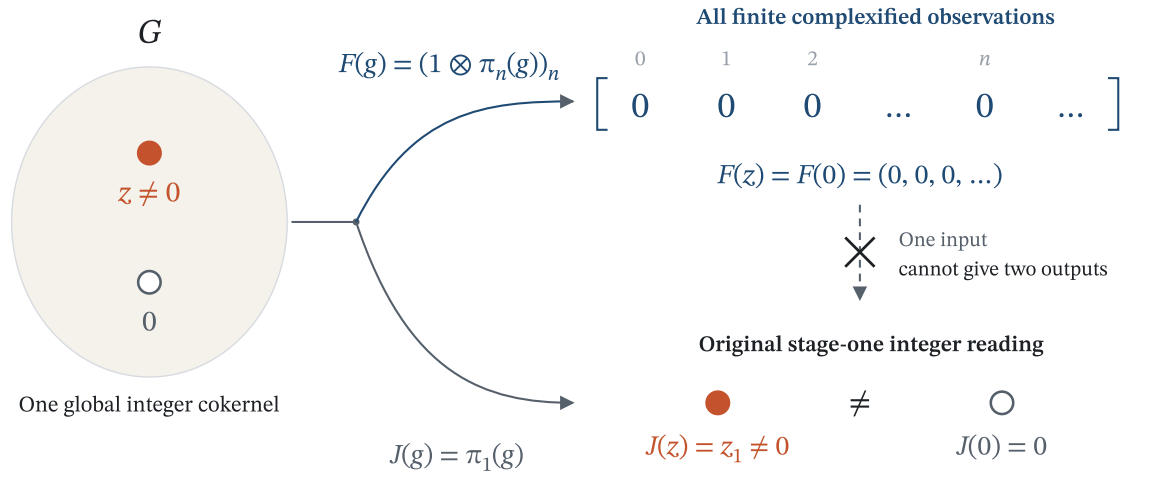
Then $z \neq 0$ and $F(z) = 0$, but $J(z) \neq 0$. Thus $\ker F \not\subseteq \ker J$. There is no function $h : \prod_n (\mathbb{C} \otimes_{\mathbb{Z}} C_n) \to C_1$ satisfying $J(g) = h(F(g))$ for every $g \in G$, nor such a factorization through the exact observable quotient $G/\ker F$.

Proof. Both facts are already established; place them on the same class. Theorem 13.1 gives $J(z) = z_1 \neq 0$, hence $z \neq 0$. Lemma 13.2 gives $1 \otimes z_n = 0$ for every n, so the entire observation family satisfies $F(z) = 0 = F(0)$. This follows by tensor elimination at each stage and does not require exchanging limits and tensor products.

If $J = h \circ F$, equality $F(z) = F(0)$ would give $J(z) = J(0) = 0$, a contradiction. The exact observable quotient also identifies z and 0, so the same argument excludes any function factorization through that quotient. This excludes more than linear decoders. ■

For an integer-linear consumer c satisfying $\ker F \subseteq \ker c$, the exact observable quotient $G/\ker F$ supplies a unique linear factorization. The original consumer J explicitly violates this condition at z: the quotient identifies z and 0, while J must distinguish them. The same observation kernel therefore determines both when factorization exists and when it fails.

The opening question—"Did the debt disappear, or just that column?"—now has a complete mathematical instance. All finite-stage complexification readings, not merely one coarse reading, fail to see the same original integer class. The original integer consumer reads it at the first stage.



$$F(z) = F(0),\ \ J(z) \neq J(0) \ \Rightarrow\ \nexists\, h : J = h \circ F$$

Figure 5 Two readings of the same nonzero integer class; vermilion tracks the nonzero class z. The upper path combines all finite-stage complex observations into F: z and the zero class give exactly the same zero family. The lower path J retains their difference. If J=h∘F, one F value would have to produce two different J values. No such function h exists.

14 Long Computations Do Not Get to Abandon Their Ledger

For every analytic input specified in this section, the old program remains on the ledger after its window value is produced. The following conditional construction puts the finite cutoff, first inverse residual, second mean, and update cochain into the next inventory together. The next correction reads its residual from this actual write, executes its own program, pays, and delivers the next packet. A window value cannot claim credit for an entire computation history.

14.1 Objects: Four Source Components, Two Actual Occurrences

The Riesz source-calculus construction supplies the analytic input for this section: the original Hilbert carrier, defect and Fourier operators, source-qualification projection, finite window, and two mean identities. We specify these objects below and prove how they actually execute, enter the complete inventory, pay their responsibilities, and generate the next correction. Let $\mathcal{H}$ be the original complex Hilbert carrier, D the original defect operator, and $\mathcal{F}$ the original Fourier operator. Use the complete four-component carrier

$$\mathcal{B} = (\mathcal{H} \times \mathcal{H}) \times (\mathcal{H} \times \mathcal{H}), \qquad \iota(f) = ((f, Df), (\mathcal{F}f, \mathcal{F}Df)), \qquad \pi((u,v),(w,t)) = u. \tag{50}$$

We have $\pi\iota = \mathrm{id}_{\mathcal{H}}$. The four components remain the source, defect, Fourier source, and Fourier defect throughout; a window operation does not rename them.

For a real parameter a, let $\mathcal{H}_a$ be the original interval carrier, $Z_a : \mathcal{H}_a \to \mathcal{H}$ zero extension, and $R_a : \mathcal{H} \to \mathcal{H}_a$ restriction. The original construction gives $R_a Z_a = \mathrm{id}$. The Pa carrier $\mathcal{H}_{\mathrm{Pa}}$ is the original even ambient space; $P : \mathcal{H} \to \mathcal{H}_{\mathrm{Pa}}$ is the original even projection and $M : \mathcal{H}_{\mathrm{Pa}} \to \mathcal{H}$ the original Möbius source operator. The compact coPoisson source generates a closed subspace $\mathcal{K}_{\mathrm{co}} \subseteq \mathcal{H}_{\mathrm{Pa}}$. Its qualification projection is $\mathcal{E} = \Pi_{\mathcal{K}_{\mathrm{co}}} \circ P : \mathcal{H} \to \mathcal{H}_{\mathrm{Pa}}$, where $\Pi_{\mathcal{K}_{\mathrm{co}}}$ is orthogonal projection. Thus the projection construction directly gives $\mathcal{E}f \in \mathcal{K}_{\mathrm{co}}$. The Tate reciprocal action $J : \mathcal{H} \to \mathcal{H}$ and source-window integral $\ell : \mathcal{H} \to \mathbb{C}$ are original continuous linear operators. The original finite cutoff list is

$$L_a = \mathrm{toList}(\mathcal{C}_a), \qquad \mathcal{C}_a = \text{the original finite centered Möbius cutoff set.}$$

This list is specified by the mathematical parameter. Each positive integer $m \in L_a$ has an actual scaled pullback $A_{a,m} : \mathcal{H} \to \mathcal{H}_a$. Let $c_a \in \mathcal{H}_a$ be the original constant interval field and define

$$W_a(f) = \sum_{m \in L_a} \frac{1}{m} A_{a,m} f - \frac{1}{2} \ell(Jf) c_a, \qquad \widehat{W}_a = \iota Z_a W_a \pi. \tag{51}$$

The operator W_a is the original finite source-window action, and $\widehat{W}_a$ returns it to the complete four-component carrier. The sum, reciprocal weights, and Tate pole term all remain in the program.

Fix a zero-observation receipt at the original analytic occurrence, and write z for its complex coordinate. The receipt carries $\zeta_{\mathrm{src}}(z) = 0$, where ζ_{src} is the analytic function generated by

that occurrence; the original analytic identification recovers the Riemann ζ. This conditional construction also requires nontriviality $z \neq -2(n+1)$ for every $n \in \mathbb{N}$ and the right-half-plane source condition $\operatorname{Re} z > 1/2$. Fix any depth, ordinal $\in \mathbb{N}$ and real parameters h, H, N, a; they index the choice of original factory, material, and occurrence.

This section proves a construction for every such input; it does not construct an inhabitant of that input type. If the Riemann hypothesis holds, every nontrivial zero has real part one half, contradicting the strict right-half-plane condition here. The input type is then empty and Theorems 14.1–14.2 hold vacuously. The proofs retain the full input-to-program, inventory, and successor contract; they do not assert a run satisfying these analytic premises.

The paid programs below start from the $S \in \mathcal{B}$ and Pa source generated by that input:

$$\nu = M\mathcal{E}\pi(S), \tag{52}$$

where $\mathcal{E}$ is the original qualified Pa projection. The qualification of ν comes from the source proof that $\mathcal{E}\pi(S)$ lies in the original compact coPoisson closed range. The original Pa program executes

$$x + \iota(M\mathcal{E}\pi(S)),$$

in the original writer. Consequently, the difference between its born environment and the total paid value of the same occurrence, followed by exact grade collapse and π, is ν. The later finite-window input $s = \iota(\nu)$ is read from precisely this written occurrence.

The same ν also pays for both means. With $q = \mathcal{E}\pi(S)$, the original position and Fourier mean identities are

$$\operatorname{mean}_{\mathrm{pos}}(q) = -\frac{1}{2}\ell(J\nu), \qquad \operatorname{mean}_{\mathrm{Fourier}}(\mathcal{F}_{\mathrm{even}}q) = -\frac{1}{2}\ell(\nu). \tag{53}$$

The first term already enters the Tate pole program in (51); the second has its own linear program and execution trace. The complete inherited material of S and the new action material remain in the original packet.

Theorem 14.1 (Every finite-window entry enters the actual write).

For each set of original inputs above, the source generates a finite-window packet p_F, its own completion receipt, complete execution inventory, and whole-ledger write. The difference Δ_F between the first actual born environment and the total paid value of the same occurrence is exactly $\widehat{W}_a(s)$, with window $W_a(\nu)$. The four original execution inventories—forward stock, inverse residual, second mean, and cochain—are all installed before birth. From this actual Δ_F, read

$$\eta = \nu - MP\pi(\Delta_F), \qquad r = \iota(\eta). \tag{54}$$

The value η equals the complete residual generated by the first inverse program. The actual window, both original means and their origins, and all execution entries jointly enter this born packet. The completion receipt of this occurrence also supplies its own writer, same-debt payment, and exact whole-ledger destination.

The original residual is retained at its full value. The parameter a and cutoff members are parameters of the source operator; syntax-node count is the charge unit here.

Proof One: Every Cutoff Term Actually Executes in the AST

Use the original language of constants, variables, addition, and linear nodes. For an environment ρ, write evaluation as ev_ρ and the remaining count as $r(e)$. The original language gives

$$\begin{aligned} r(\operatorname{var} x) &= 1, & r(\operatorname{const} v) &= 0, \\ r(e+f) &= r(e)+r(f)+1, & r(Ae) &= r(e)+1. \end{aligned} \tag{55}$$

Its canonical execution supplies $T : e \to \operatorname{const}(\mathrm{ev}_\rho\, e)$, retaining every intermediate expression and execution step, with $|T| = r(e)$. Write $\delta_e = [e]$ for the basis generator indexed by the complete expression e in the free integer word module. Each step $e_j \to e_{j+1}$ generates the ordered entry $\delta_{e_j} - \delta_{e_{j+1}}$. The number of entries therefore equals $|T|$, and their sum telescopes to $\delta_e - \delta_{\operatorname{const}(\mathrm{ev}_\rho\, e)}$. These shared execution facts are reused by name from Proposition 3.7 and Supplement S0.1–S0.3 of the process-core second edition [6].

On $\mathcal{B}$, define the actual linear nodes

$$U_{a,m} = \iota Z_a A_{a,m} \pi, \qquad B_m = \frac{1}{m}\operatorname{id}_{\mathcal{B}}, \qquad Q_a = \iota Z_a (c_a\,[-\tfrac{1}{2}\ell J])\pi.$$

The AST of cutoff term m is $B_m(U_{a,m}(\operatorname{var} x))$. Put $e_{a,[]} = \operatorname{const} 0$ and define recursively

$$e_{a,m::L} = B_m(U_{a,m}(\operatorname{var} x)) + e_{a,L}, \qquad e_a = e_{a,L_a} + Q_a(\operatorname{var} x). \tag{56}$$

Induct on the list. Both sides vanish for the empty list; at $m :: L$, addition evaluation adds the new actual weighted pullback to the previously computed sum. Thus, for every environment ρ,

$$\mathrm{ev}_\rho(e_{a,L}) = \sum_{m\in L} B_m U_{a,m}(\rho(x)), \qquad \mathrm{ev}_\rho(e_a) = \widehat{W}_a(\rho(x)). \tag{57}$$

Put $\rho_s(x) = s$. If the actual received execution trace is $T : e_a \to \operatorname{const} u$, its soundness gives $\widehat{W}_a(s) = u$. Apply $\pi\iota = \mathrm{id}$ and $R_a Z_a = \mathrm{id}$ to obtain

$$R_a \pi(u) = W_a \pi(s) = W_a(\nu). \tag{58}$$

The proof reads the actual result from the received trace; the window value is not a construction input. Let $|L_a| = k$. Each cutoff term has count 3, each adjoining addition costs 1, the pole term has count 2, and final combination costs 1. Hence $r(e_a) = 4k + 3$. This counts the nodes of this language exactly, without a separate accounting of the linear primitives' internal costs.

Proof Two: Install Four Inventories; Let the Original Receipt Write the New Window

Define the original first inverse action $I = \iota M P \pi$. Its literal residual program is

$$e_a^{\mathrm{inv}} = \operatorname{var} x + (-I)e_a.$$

By (57) and $\pi\iota = \mathrm{id}$, its first source component in ρ_s is

$$\pi \,\mathrm{ev}_{\rho_s}(e_a^{\mathrm{inv}}) = \nu - MPZ_a W_a(\nu). \tag{59}$$

The first inverse therefore retains the complete residual and the exact equality source = reconstructed source + residual.

To write into the old inventory, use the program

$$f_{a,s} = \mathrm{var}\, x + e_a[x := \mathrm{const}\, s]. \tag{60}$$

The substitution-evaluation law gives, for every ρ,

$$\mathrm{ev}_\rho(f_{a,s}) = \rho(x) + \widehat{W}_a(s). \tag{61}$$

Write the original graded carrier's program lift as $e \mapsto e^{[g]}$ and its exact collapse as c_g. It adds the two coordinates of each paired value grade by grade and satisfies

$$c_g \,\mathrm{ev}_\rho(e^{[g]}) = \mathrm{ev}_{c_g\rho}(e). \tag{62}$$

Induct on g. At grade zero the map is the identity. At the next grade, pair evaluation supplies the old value and actual effect; their sum is the original evaluation at the updated environment, to which the induction hypothesis applies. Every original component is first retained in the graded carrier and only then read through the specified collapse. Distinct occurrences are not identified by equality of values.

At the actual frame ϕ, put

$$v_\phi = \mathrm{oldPaid} + \mathrm{nativeAt}(\phi, o_\phi), \qquad b_\phi = c_g(v_\phi).$$

The inventory of this occurrence is generated by the four exposure trees

$$\begin{aligned} T_F &: \ f_{a,s}^{[g]} \longrightarrow \mathrm{const}\,\mathrm{ev}(f_{a,s}^{[g]}), \\ T_I &: \ (e_a^{\mathrm{inv}}[x := \mathrm{const}\, s])^{[g]} \longrightarrow \mathrm{const}\,\mathrm{ev}(\cdot), \\ T_M &: \ \left[\iota Z_a c_a(-\tfrac{1}{2}\ell\pi)(\mathrm{const}\, s)\right]^{[g]} \longrightarrow \mathrm{const}\,\mathrm{ev}(\cdot), \\ T_B &: \ B(f_{a,s}; b_\phi, \widehat{W}_a(s))^{[g]} \longrightarrow \mathrm{const}\, 0. \end{aligned} \tag{63}$$

Every trace here uses the same actual environment writer. These traces are generated at the actual frame ϕ; below we write $T_F(\phi), T_I(\phi), T_M(\phi), T_B(\phi)$. Each δ subscript in the cochain is the complete syntax obtained by the indicated old-value or updated-value substitution; the mixed terms are generated by the AST's increment recursion. The original update cochain B takes, for any expression f,

$$\delta_{f(\mathrm{old+increment})} - \delta_{f(\mathrm{old})} - \sum_{t \in \mathrm{mixedTerms}(f)} \delta_t \tag{64}$$

expands it through the original integer-coefficient program, and binds the old value and increment to b_{ϕ} and $\widehat{W}_a(s)$, respectively. The original mixed-term update identity generates the zero evaluation of (64) node by node. The cochain program retains its own execution trace: 0 is its completion value, while its entries enter the inventory step by step.

Write $\mathsf{Exp}(T)$ for the exposure retaining the relations of every execution step and $\odot$ for the original parallel seed. The first inventory is

$$I_F(\phi) = \mathsf{Exp}(T_F(\phi)) \odot (\mathsf{Exp}(T_I(\phi)) \odot (\mathsf{Exp}(T_M(\phi)) \odot \mathsf{Exp}(T_B(\phi)))). \tag{65}$$

The parallel seed's trace consists of a new root followed by the ordered concatenation of both original traces; repeated payloads are not converted into a set. We have $\mathsf{preserve}(\text{none}, I) = I$ and $\mathsf{preserve}(\text{some } I_0, I) = I_0 \odot I$. Any original prior inventory is therefore retained together with these four new exposures. Write ϕ_F for the frame selected by the first window receipt; its four-trace inventory is $I_F(\phi_F)$.

Install the exact graded program (60) in the decoding writer, and install (65) in the original next-inventory callback before the emitter. On receiving completion occurrence o, the decoding writer reads oldPaid + nativeAt(ϕ, o) and executes that program. Both configuration equalities hold by construction.

The original completion recursion follows only actual paid branches. The remaining count decreases strictly each time, generating a Type-valued receipt $(d, v, \text{settled}, \text{action equality})$, where d is at most the initial remaining count. The action of the frame actually selected by the original receipt is precisely the receipt's mathematical settlement branch. Consequently, nativeEventValue equals the receipt's own completion value v, rather than an externally supplied value. Put $u_F = \text{oldPaid} + v$. The original nextBorn active environment equals exactly the decoding writer's write on u_F.

Equations (61)–(62) give, for every actual variable name x in the original language,

$$c_g(\text{activeEnv}_F(x)) = c_g(u_F) + \widehat{W}_a(s).$$

Subtract the same $c_g(u_F)$ to obtain

$$\Delta_F := c_g(\text{activeEnv}_F(x)) - c_g(u_F) = \widehat{W}_a(s). \tag{66}$$

The actual received execution trace can therefore rewrite its original endpoint directly as Δ_F; applying (58) gives the new window. Substitute (66) into (54): equation (59) then identifies the received inverse residual with the first residual generated by the original program. Substituting the same write equality into the actual old-value/increment environment leaves the original update cochain evaluated at zero. The dependent identification of same-debt payment and exact whole destination appears in the shared Proof Four below. This proves Theorem 14.1 and establishes that the η used by the next correction comes from this actual write.

Theorem 14.2 (Old traces and six execution families accompany actual correction, payment, and every permitted future).

For the actual p_F, s, r generated by Theorem 14.1, the next source executes the literal $s - r$ and substitutes that correction program into the original complete window program. It generates a new correction packet p_C with its own completion receipt, inventory, and whole-ledger write. The second actual write difference is

$$\Delta_C = \iota Z_a W_a(\nu - \eta). \tag{67}$$

Read η from the completion write of p_F itself, then take that packet's original receiver and nextSeed and install the correction writer and inventory before its subsequent native occurrence. This construction's own completion receipt generates p_C. The second execution thus follows the first actual packet. Proof Six identifies its root, current, and source seed field by field.

The four original traces of the first occurrence enter the new packet through the source's migration of the prior inventory. The correction frame ϕ_C also generates its own four window-execution families, a new stock trace, and a correction trace. The old traces and these six new execution families retain their respective occurrence indices. The old-write proof, new-write proof, new whole-ledger payment object, original environment/AST identities, and actual traces are eliminated in one consumer, giving

$$R_a \pi(\Delta_C) - W_a(\nu) = -W_a(\eta). \tag{68}$$

For any original Pa test vector b, the same consumer gives the complete source price

$$\langle b, P\pi(\Delta_C)\rangle = \langle b, PZ_a W_a(\nu)\rangle - \langle b, PZ_a W_a(\eta)\rangle. \tag{69}$$

The actual correction of this occurrence also generates the complete joint current/next observation fibre. The target root of each birth program is heterogeneously equal to the root of its complete born packet; the next current of the target answer-and-next equals that packet's current. Continuation starts from the packet's actual source seed. For every permitted finite future word, every future admission ordinal, and every target site, the original request and its paid boundary have equal readings.

Proof Three: The Next Execution Starts from the Received Residual

The correction AST is

$$\kappa_{s,r} = \operatorname{const} s + (-\operatorname{id})(\operatorname{const} r). \tag{70}$$

It has one linear node and one addition node, with count 2, and generates its own complete execution trace. Nodewise evaluation gives $\operatorname{ev}(\kappa_{s,r}) = s - r$. The new window and stock programs are, respectively,

$$g_{a,s,r} = e_a[x := \kappa_{s,r}], \qquad j_{a,s,r} = \operatorname{var} x + g_{a,s,r}. \tag{71}$$

Apply the substitution-evaluation law and then (57). For every environment ρ,

$$\mathrm{ev}_\rho(g_{a,s,r}) = \iota Z_a W_a \pi(s-r), \qquad \mathrm{ev}_\rho(j_{a,s,r}) = \rho(x) + \iota Z_a W_a \pi(s-r). \tag{72}$$

The new writer executes $j^{[g]}$ in the same source environment, with new inventory

$$I_C(\phi_C) = I_F(\phi_C) \odot (\mathsf{Exp}(T_C^{\mathrm{stock}}) \odot \mathsf{Exp}(T_C^{\mathrm{correction}})), \tag{73}$$

Here $I_F(\phi_C)$ reexecutes the four window-program families using the correction frame's own original environment and paid material. The first occurrence's four actual traces belong to $I_F(\phi_F)$; an order-preserving typed migration of the original inventory carries them into Prior_C, retaining their entries and original lengths. The complete born stock is $\mathsf{preserve}(\mathrm{Prior}_C, I_C(\phi_C))$, so old traces and this frame's new traces coexist. The two additional traces execute $j^{[g]}$ and $\kappa^{[g]}$, respectively; their exposures retain every step in original order. The variables in every cutoff term and pole term of e_a are actually replaced by the same κ. The entire composite program therefore has count $5k+4$ and the stock program count $5k+6$; repeatedly called nodes are counted with their actual occurrence multiplicities.

Install the new inventory and decoding writer before the original emitter. Obtain this new configuration's own completion receipt, receipt-selected frame, and nextBorn. Repeat Proof Two's receipt-value and graded-collapse derivations. Taking u_C to be the oldPaid + v_C of this same occurrence gives

$$\Delta_C := c_g(\mathrm{activeEnv}_C(x)) - c_g(u_C) = \mathrm{ev}_{\rho_s}(g_{a,s,r}) = \iota Z_a W_a \pi(s-r). \tag{74}$$

Since $s = \iota(\nu)$ and $r = \iota(\eta)$, equation (67) follows. The actual received g-trace identifies this endpoint by (74). Apply trace soundness, $\pi\iota = \mathrm{id}$, $R_a Z_a = \mathrm{id}$, and linearity to obtain

$$R_a \pi(\Delta_C) - W_a(\nu) = W_a(\nu - \eta) - W_a(\nu) = -W_a(\eta),$$

which is (68). The same write equality eliminates the actual update cochain at $(c_g(u_C), \Delta_C)$, again generating the exact zero-valued relation and complete execution inventory.

Proof Four: One Consumer Takes the Old Write, New Payment, and New Trace

Each window whole-ledger object carries the complete dependent data: a source-generated same-debt payment p; a proof that p.step equals the actual debt step; the same ledger root, temporal visit, and responsibility row; heterogeneous equality between the payment evolution and the original whole-ledger destination; original responsibility authority; and proofs identifying the born registered environment and AST with the original request environment and AST of that paid material.

The payment is generated by the syntax credit of the original residual request. Birth first executes its initial step, leaving exactly the original request count minus 1. The budget identity of the original residual AST makes the remaining credit strictly positive. A mathematically settled born action would have zero budget, contradicting that positive credit, so the original action takes its actual paid branch. The original same-debt lineage turns this execution step into a payment:

actual one-step count decrease gives strict debit, while the debt step's budget monotonicity gives no refill. The original Inquiry and Restructuring visit presentations agree by induction on the count; hence payment and birth have exactly the same ledger root, temporal visit, and row. Transport both ledger evolutions of the original whole write and payment along the same current identity and apply destination to the same responsibility row, obtaining equality of the whole destinations. These are direct uses of this paper's same-debt payment construction in the present instance.

The first consumer passes $(\mathrm{whole}_F, \mathrm{write}_F, \mathrm{Trace}_F)$ to the supplied function, and the second passes $(\mathrm{whole}_C, \mathrm{write}_C, \mathrm{Trace}_C)$. The actual composite consumer follows the dependent order

$$\mathrm{consume}_F(\lambda \mathrm{whole}_F, \mathrm{write}_F, \mathrm{Trace}_F.\mathrm{consume}_C(\lambda \mathrm{whole}_C, \mathrm{write}_C, \mathrm{Trace}_C.\ \cdots)). \tag{75}$$

The outer write_F is eliminated into (54), identifying the actual residual's source. The inner Trace_C is eliminated into (68), identifying the actual window change. The inner write_C is eliminated into (74), identifying the complete new source field. At the same time, whole_C supplies actual payment, the exact ledger row and whole-ledger destination, and source environment and AST. Every object retains its own occurrence identity and continues through the exact packet.

Finally, apply P and $\langle b, - \rangle$ to the complete field supplied by write_C. Three linear maps preserve the difference, giving (69). This step actually consumes the old write and new execution trace; it accepts no additional desired window price.

Proof Five: Joint Observation Retains the Correction's Current and Next Fibres

On the complete $\mathcal{B}$, define

$$\mathcal{R}_a = \mathrm{id} - I\widehat{W}_a, \qquad \mathcal{A}_a = \mathrm{id} - \mathcal{R}_a, \qquad O_a = W_a \pi. \tag{76}$$

By (59), $\mathcal{R}_a \iota(\nu) = \iota(\nu - MPZ_a W_a \nu)$. The outer old write also gives $r = \mathcal{R}_a(s)$, so the actual received correction satisfies

$$\mathrm{ev}(\kappa_{s,r}) = s - r = \mathcal{A}_a(s). \tag{77}$$

Take the joint observation

$$J_a(x) = (O_a x, O_a \mathcal{A}_a x), \qquad \mathcal{Q}_a = \mathcal{B} / \ker J_a. \tag{78}$$

If $x \in \ker J_a$, both components vanish. Hence $x \mapsto [x]_{O_a}$ and $x \mapsto [\mathcal{A}_a x]_{O_a}$ are well-defined on $\mathcal{Q}_a$, giving $\mathsf{current}$ and next, respectively. Thus

$$\mathsf{current}([s]_{J_a}) = [s]_{O_a}, \qquad \mathsf{next}([s]_{J_a}) = [\mathcal{A}_a s]_{O_a} = [\mathrm{ev}(\kappa_{s,r})]_{O_a}. \tag{79}$$

The joint quotient classes of any x, y are equal exactly when $J_a(x - y) = 0$. Componentwise this is

$$O_a x = O_a y \quad \text{and} \quad O_a \mathcal{A}_a x = O_a \mathcal{A}_a y. \tag{80}$$

This retains the complete joint current/next observation fibre. The final equality in (79) also uses the actual residual generated by the old write, so this quotient's next reader directly reads the actual correction of this occurrence.

Proof Six: Birth Root, Current, Actual Source Seed, and Every Future Reader

Write a complete admission packet as $p = (g, (\phi, \xi), C, \lambda)$, with $\lambda : C.\mathrm{LowVar} = X$. The native window packet first retains the original receiver, nextSeed, newly installed emitter, and complete inventory at the exact next grade. The original bornAfterNative takes nextBorn from its own completion receipt and transports it along λ to form p_F. The correction packet p_C uses the same construction with the new emitter and decoding writer of (73).

For every receipt-selected frame ϕ_*, the original birth program generates a target at its exact temporal causal event o_*. The two original compilation equalities give

$$\begin{aligned} \mathrm{birth}(o_*).\mathrm{targetRoot} &= \mathrm{root}(\mathsf{nextBorn}(\phi_*, C_*)), \\ \mathrm{birth}(o_*).\mathrm{answerAndNext.nextCurrent} &= \mathrm{current}(\mathsf{nextBorn}(\phi_*, C_*)). \end{aligned} \tag{81}$$

Packet transport does not change these readers. Eliminating the language equality reduces them to heterogeneous reflexivity and ordinary reflexivity, respectively. Composing (81) with the exact born-packet construction directly gives the claimed birth-root/packet-root and birth-current/packet-current identifications. The complete birth occurrence is identified here; no equal window value is used to jump to root identity.

Continuation shifts the original factory by the packet's actual grade. After k further grades, its new value family is exactly the original family's grade $g + k$. This follows by induction from gradewise pairing recursion and $g + (k + 1) = (g + k) + 1$. The same equality transports all five original callbacks: datum, scalar inventory, pair inventory, extra-scalar, and extra-pair.

The first continuation step then binds all five callbacks to the ξ belonging to p itself. For any auxiliary source seed ζ, its first-step configuration, scalar, pair, receiver, nextSeed, and nextBase equal the corresponding objects of the original shifted factory at ξ. The equalities for datum, the two inventories, and the two extras hold by definition. Receiver and nextSeed are read back from these equalities. NextBase also uses the same original writer, whose equality holds variable by variable. An auxiliary source seed therefore cannot replace the actual origin, and the complete first-step callbacks are retained in later births.

Finally, use the original permitted action S on the complete word carrier $\mathcal{W} = \bigoplus_j \mathcal{W}_j$ and the site readers D_t. Theorem 9.12 and Supplement S5.4 of the process-core second edition [6] construct the maximal future-invisible kernel

$$K_{\mathrm{fut}} = \bigcap_{w \in \{*\}^{<\omega}} \bigcap_{t} \ker(D_t S_w). \tag{82}$$

Write q_{j+1} for the request generated by completion occurrence j and registered in its next packet. This paper's original-root residual consumer proves that its complete AST word and the paid boundary b_{j+1} of that occurrence have the same model class after inclusion at the

corresponding site. Apply this to the continuation from the actual source seed above and expand the equality-of-fibres identity. For all $j \in \mathbb{N}$, $w \in \{*\}^{<\omega}$, and $t \in \mathbb{N}$,

$$D_t S_w(\iota_{j+1}[q_{j+1}.\mathrm{AST}]) = D_t S_w(\iota_{j+1} b_{j+1}). \tag{83}$$

Every permitted future reader of both the finite-window packet and correction packet thus continues to consume actual paid material and the next request. Pair the objects of Proofs One through Six at the same actual packet to obtain the full conclusion: both whole ledgers, actual residual, same-occurrence payment and source price, joint fibre, birth root/current, actual source seed, and all permitted futures. □

15 The Exact Connections to Existing Methods

15.1 Mathematical Accounting: The Algebra of Differences and the Identity of Responsibilities

Ellerman's mathematical double-entry bookkeeping identifies pairs of nonnegative numbers by equality of cross-sums: $[x\,//\,y] = [u\,//\,v]$ if and only if $x + v = y + u$. Coordinatewise addition supplies a group structure. This construction explains how the debit and credit sides express signed differences without negative numbers and extends to vector accounting. [1] Cancellation of equal amounts on both sides is precisely equality in this difference group.

Whole-ledger accounting adds contracts for complete event admission, original responsibility identity, accepted transfers, parentwise discharge coverage, and the absence of same debt in the actual successor. Equality of number pairs concerns a reading. Equation (44) is a dependent proposition about the original terminal receipt and the complete responsibility fibre in the actual successor. Both may hold, or the first may hold while the second cannot be constructed. Section 4 gives the exact condition under which a difference reader suffices for its designated consumer.

15.2 Proof-Carrying Code: Who Checks the Proof, and What Does It Bind?

In Necula and Lee's proof-carrying code, the receiver specifies a safety policy and a method for generating verification conditions, while the code provider supplies a proof. From the code actually received, the receiver regenerates the verification conditions and checks the proof to decide whether to execute it. [2] The code and the verified proposition thereby jointly determine eligibility for execution.

Proof-carrying code binds a program to the safety proof required by its receiver. Whole-ledger accounting binds disposition to the source occurrence, complete event inventory, complete ledger writeback, original causal entry, and actual successor. The disposition in (4) retains the original event index; settlement in (44) jointly checks the original terminal receipt and the original process's exact successor. Complete inventory admission, entrywise destinations, and two-condition settlement thus form an independently checkable contract. Execution conforming to a given safety policy and settlement of an original debt are different propositions.

15.3 Resource Logic: The Frame Rule Does Not Call the Final Curtain

Reynolds's separation logic uses separating conjunction to express disjoint storage resources and the frame rule to support local program reasoning. [3] A finite set of explicit rows plus an exact remainder in whole-ledger accounting serves a related operational need: a tick need not rewrite the entire ledger row by row, but the omitted part must have a definite semantics. An unchanged remainder is exact identity; a transported remainder is the whole action of an original source-generated event. “Untouched” cannot mean “gone.”

Atkey combines consumable resources with separation logic, using local resource annotations for amortised resource analysis. His safety theorem tracks both resources consumed and resources required in the future, and at termination supplies the postcondition and remaining resources. [4] This paper's natural-number decrease has a clear affinity with that resource discipline, but tracks the same responsibility across actual transfers. Main Theorem 10.3 obtains well-foundedness only from actual strict payment; Main Theorem 11.4 separately uses a total emission law. The appearance of a budget gives no free termination, and zero budget gives no free terminal receipt.

Responsibility incidences are not assumed to form a separated heap. Discharge requirements may be related, and splits and merges must explicitly cover those relations. The original source's transfer, split, and merge receipts determine which responsibility combinations are lawful.

15.4 Provenance Semantics: After the Polynomial, Follow the Next Tick

Green, Karvounarakis, and Tannen annotate relational tuples with semirings and organize the commutation of positive relational algebra with annotation-semiring homomorphisms. Universality of the polynomial semiring makes each concrete provenance interpretation factor through a unique homomorphism. [5] A provenance polynomial records which inputs and combinations produced a result.

The additional connection addressed by whole-ledger accounting is dynamic responsibility: whether two rows with the same origin belong to the same original debt, which action actually changed it, and whether it continues in the exact successor after settlement. Its source indices also bind admission authority, acceptance, discharge jurisdiction, original compiler branch, and complete target ledger. Section 13 gives a consumer-factorization criterion through the observation kernel and uses an actual nonzero class to show that the original integer consumer fails it. The consumer about to use an origin must determine what provenance needs to be retained.

These connections now also carry positive continuation. Section 9 generates writable syntax, actual same-debt payment, and a completed answer from the original residual; Section 14 carries the old execution inventory into the next correction. The contracts preserve complete provenance, disposition, and the next occurrence entry by entry. Their constructibility is supplied by the commuting identities of the same original program and its direct readers.

16 The Ledger Does More Than Hold a Grudge. It Pays.

The opening zero is still there. It now has an inspectable actual history beside it.

The complete event inventory does not prune continuation branches, and the complete open fibre does not lose an entry because a report leaves a blank. Original admission identity, parent-and-child coverage, and exact transported remainders carry responsibilities through moves, splits, and merges. One actual payment of a fixed original debt, together with two finite non-refilling histories, generates a strictly decreasing macrostep relation. The original terminal receipt and absence of same debt in the successor then supply final settlement. Original target six delivers these objects entry by entry; the complete integer consumer, meanwhile, prevents an entire family of zero observations from signing for a nonzero original class.

The original ledger also performs the action it could not perform at the opening. From the original residual of the complete future model, it grows the next request; from its own syntax, it generates actual same-debt payment; through the same suffix, it completes the original input; and from that completion event, it delivers an answer, original receipt, and complete ledger destinations. The old responsibility is not abandoned. A new responsibility is born with its own source, and the original canonical successor continues consuming all paid material. Under the input contract of Section 14, the finite window's four old traces enter the correction program, and two new traces and the actual new inventory jointly enter the next occurrence. The residual becomes the next program's input.

The balance is zero. Where is the settlement receipt? The original ledger can produce it. If this responsibility must continue, it can also produce the next request, actual payment, and the next complete ledger.

You may still owe. You may not pretend you paid. The ledger may continue, but it has to pay for itself.

Appendix A Shared Theorems and This Paper's Independent Proofs

The process-core second edition [6] provides the following shared objects and complete proofs. This paper retains those theorem identities and independently proves whole-ledger disposition and the new actual consumers.

Location here	Shared result reused by name	Proof undertaken here
§2–3	Complete root, temporal visit, canonical successor; Proposition 6.1	Complete inventory admission, three fragment-coverage forms, dependent row authority, and actual entrywise disposition
§5, §7	Proposition 3.7; Theorem 6.10, Supplement S0–S1	Split/merge lifts on arbitrary Type inventories, original whole-ledger remainders, independent syntax responsibility, and coefficient/binding/call charges
§8.1–8.2	Theorems 7.7, 8.12, 9.5–9.9; Supplements S2, S4, S5.1	Complete inventory elimination, order preservation at each occurrence, admission of new material, and actual writes of next content
§8.3	Theorems 6.12, 9.10–9.12; Supplements S1.3, S3.3, S5.3–S5.4	Every segment of the same runtime, complete material transport, and actual succession of paid inventory
§9	The execution and model foundations above	All-environment residual identity, original-root writable requests, same-debt payment destinations, complete suffixes, and same-occurrence answer/receipt/whole ledger/successor
§10–11	The two levels of Theorem 6.7	Separate proofs of payment well-foundedness without a total law and finite settlement with a total law at each current; successor same-debt checks
§12	Theorems 8.4, 8.6–8.7; Proposition 8.9	Complete old-ledger/new-row disposition at the same first write and exact splits in richer revisions
§13	Original unit-arithmetic root and complete representation interface	Prime-three nonzero coordinate, determinant annihilation, and failure of the complete finite complex observation family to recover the original integer consumer
§14	Original executor, paired language, future kernel of Theorem 9.12	Four finite-window traces, two literal-correction traces, nested same-occurrence write consumption, actual seed, and every permitted future

Sections 9 and 14 give the new proofs. The supplement covers Type inventories, charges, admission and cofinal continuation in Sections 5, 7 and 8.

Appendix B Contracts and Units

Object	Original objects that it must consume
Faithful termination	Complete event presentation and independent ordinary-outgoing-event exhaustion proof
Entrywise disposition	The same whole-ledger compilation, generated fragment, and exact remainder
Same-debt payment	Original actual step, unique same-debt target, and strict budget debit
Finite settlement history	Payment well-foundedness plus a total law emitting exact settlement or actual payment at each current
Final settlement	Original terminal receipt plus an empty complete same-debt fibre in the actual successor
Syntax completion	Every node of the original AST, complete paid trace, and constant endpoint
Completion answer and continuation	Original completion action, its answer and receipt, complete ledger write of the same occurrence, and canonical successor
New responsibility	Its own actual source, first birth step, and independent claim branch

Syntax charge is measured in original execution nodes; same-debt budget is read from the original open witness. Mathematical completion, same-debt payment, and domain termination each use their own original receipts. They agree within the same actual chain without becoming the same numerical quantity. “Unforgeable” in the dependent type semantics is defined in Section 1; actual termination and budget units are read from their respective source contracts.

Code and data availability

The complete WA1–WA40 formal proofs, direct consumers, resources and reproduction programs are bound to H0mework commit `4301fa8c5bb19bb1e8c3824d7c69be406264ed12`. The per-claim map records original source epochs, producer and consumer owners, complete declaration types and acceptance receipts; the fixed material commit is `f607eb646d4e9cddba3512755365842895af44ca`. The 13 historical proof packages retain separate Lean environments and their exact entry modules, including `H0mework.Papers.WholeLedgerAccountingRba7e274b` for WA38–WA40. The third-stage payment example in Section 11.7 uses the Y1 proof repair without changing its proposition; the remaining Y material retains its original epoch.

The reproduction guide fixes Lean 4.33.0 and Mathlib v4.33.0. From the repository root, run `make check-whole-ledger-entry` to recover the source and resource selection; prepare dependencies with `make bootstrap` and run `make check-whole-ledger-full` for the full import build, metadata trust and complete-type checks. The acceptance record binds their actual results. Complete written proofs accompany this paper; shared process results use the process-core second edition, DOI 10.5281/zenodo.23297175. Editable vector figures and generators are included. Original code uses Apache-2.0; the paper and original figures use CC BY 4.0.

References

[1] David Ellerman. *On Double-Entry Bookkeeping: The Mathematical Treatment*. Accounting Education, 23(5), 483–501, 2014. DOI: 10.1080/09639284.2014.949803. Author's preprint.
[2] George C. Necula and Peter Lee. *Safe Kernel Extensions Without Run-Time Checking*. Proceedings of OSDI, 229–243, 1996. DOI: 10.1145/238721.238781. USENIX paper.
[3] John C. Reynolds. *Separation Logic: A Logic for Shared Mutable Data Structures*. Proceedings of LICS, 55–74, 2002. Author's paper.
[4] Robert Atkey. *Amortised Resource Analysis with Separation Logic*. Logical Methods in Computer Science, 7(2:17), 2011. DOI: 10.2168/LMCS-7(2:17)2011. Author's paper.
[5] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. *Provenance Semirings*. Proceedings of PODS, 2007. Author's paper.
[6] Jian Gao. *The State Is Not the History, the Source Is: Identity, Responsibility, and Minimal Revision in Processes*. Second edition, 2026. DOI: 10.5281/zenodo.23297175. Main text and complete proof supplement.